

课程笔记

Harness Engineering: 从 Claude Code 泄露源码看 Agent 工程的下一个范式

智猩猩 / 李博杰 & Codex

2026 年 5 月 25 日



视频作者/频道: 智猩猩

发布日期: 2026-05-20

视频时长: 00:25:28

视频链接: <https://www.bilibili.com/video/BV1G2Lz6uEHY/>

目录

1 引子：从源码泄露看工程深度	3
1.1 Claude Code 泄露为何值得分析	3
1.2 OpenCode 与 Claude Code 的差异：概念原型与生产级产品	4
1.3 为什么“办事不靠谱”常来自 Harness 缺口	5
1.4 本章小结	5
2 Harness Engineering: Agent 能力的上限与下限	6
2.1 从 Prompt 到 Context 再到 Harness	6
2.2 模型、Observation space 与 Action space	6
2.3 Harness 的三类动作：约束、验证、纠正	8
2.4 Prompt caching、上下文压缩与 side query	8
2.5 Markdown 文件系统记忆、vector database 的边界与 dream 机制	10
2.6 本章小结	12
3 生产级 Agent 的安全、权限与错误恢复	12
3.1 Agent 致命三要素	12
3.2 敏感信息读取与外发的双向防护	13
3.3 错误恢复：从工具漏调到输出中断	14
3.4 Sub-agent 工具集合与角色边界	15
3.5 语义命令解析优于关键词名单	16
3.6 本章小结	17
4 用研究方法做产品：Evaluation、Ablation 与反蒸馏	17
4.1 Frontier lab 为什么把产品当实验系统做	17
4.2 Evaluation 体系与 prompt 快速迭代	18
4.3 Ablation baseline、GrowthBook 与工程化实验	19
4.4 Anti-distillation: fake tools、训练集投毒与 CoT 摘要	20
4.5 本章小结	22
5 行业判断：GUI、Context 与 AI-native Organization	22
5.1 GUI 作为人的注意力补丁	22
5.2 软件价值的迁移：数据、业务逻辑、权限和治理	23
5.3 Context 决定人和 Agent 的可行动范围	24
5.4 AI-native organization、Brooks 定律与概念完整性	25
5.5 三类更稳固的人类专业原型	27
5.6 一人公司的误区	28
5.7 本章小结	28
6 核心公式：Model × Harness = Agent	29
6.1 公式的含义：协同优化	29
6.2 用户问题、应用层 bug、模型训练与 Harness 兜底	30
6.3 First-party data 与数据飞轮	32
6.4 Harness 化石与长周期任务的新问题	32

6.5	本章小结	33
7	总结与延伸：模型商品化与应用层护城河	34
7.1	顶尖模型与中端模型的分化	34
7.2	API 成本下降规律与短期技术杠杆	35
7.3	应用层护城河为什么还要放在技术之外	36
7.4	全文综合：从源码细节到组织形态	36
7.5	后续问题与实践清单	37

1 引子：从源码泄露看工程深度

这场演讲从一个很具体、也很适合作为工程样本的事件切入：Claude Code 的源码在演讲前一个月左右泄露。讲者没有把它当成八卦来讲；他借泄露代码打开一张剖面图：一个成熟 Agent 产品，究竟把多少能力放在模型之外，多少能力沉淀在架构、流程、权限、记忆和恢复机制里。

开头几分钟里，讲者先点出三个对象：近期很火的 OpenCode、Claude Code，以及马上就要展开的 Harness Engineering。这里的 ASR 把 OpenCode 识别成了“Open Cloud”，把 Claude Code 识别成了“Cloud Code”，后文统一采用技术语境中更合理的写法。这个校正很关键，因为本节讨论的核心正是两类代码产品的对照：一个更像快速做出来的通用 Agent 框架，另一个经过团队、用户反馈和商业化运行反复打磨。

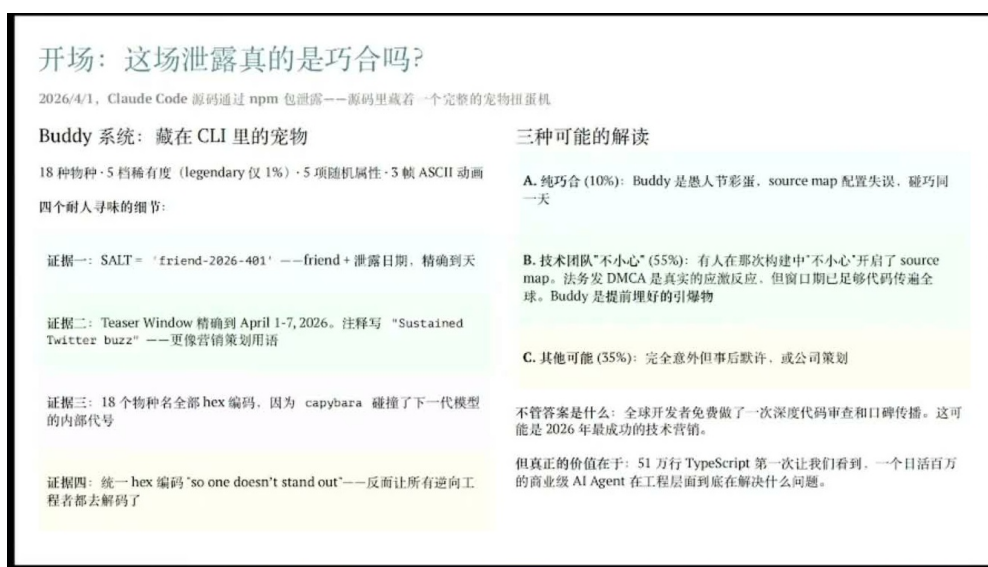


图 1: Claude Code 源码泄露被拆成若干证据线索：Buddy 系统、时间窗口、注释与内部代号共同构成开场案例。¹

1.1 Claude Code 泄露为何值得分析

讲者对源码泄露的第一层判断很直接：这件事未必只是偶然事故。理由来自若干源码细节，例如大量注释、“sustained twitter fuzz”这类带有传播感的文本，以及一些物种名与后续模型内部代号发生碰撞的痕迹。讲者据此提出一个可能性：这也许包含技术营销成分。这个判断本身无须过度坐实；更有价值的是，它让读者意识到，泄露源码一旦进入公共视野，外界看到的就不只是功能表，还能看到产品背后的工程取舍。

如何读一次源码泄露

源码泄露的教学价值不该停在“拿到秘密”的兴奋上，更该落到真实产品如何处理粗糙边缘：哪些地方写了注释，哪些地方做了兜底，哪些地方为了成本、速度、权限和用户反馈留下了专门路径。对 Agent 产品而言，这些边缘往往比主循环本身更说明问题。

这也是本章的入口：如果只看演示视频，OpenCode 和 Claude Code 都能调用模型、读写代码、执行任务；如果看源码和使用表现，差异会落到更低层的工程组织。类比成汽车，模型像发动机，外

¹ 视频画面时间区间：00:00:18–00:00:55。

层工程像传动、刹车、仪表、保险丝和维修手册。发动机功率再高，刹车和传动不稳，驾驶体验仍会飘，甚至危险。

1.2 OpenCode 与 Claude Code 的差异：概念原型与生产级产品

讲者给出的对照很锋利。OpenCode 是一个非常通用的 Agent 框架，带有强烈的产品想象力：它对接了很多外部 channel，用户交互有“活人感”，插件可以通过自然语言安装和配置，还推崇“Skills + CLI”的组合。这些设计在交互层面很新，尤其是自然语言配置插件这一点，把传统“用户去翻配置文件”的步骤，改成了“用户把意图说出来，Agent 自己改代码和配置”。

讲者在 00:01:58-00:02:17 还给了一个更准确的判断边界：如果问 OpenCode 会不会成为未来 Agent 的操作系统，他认为难度很高；但这不等于 OpenCode 没有产品价值。OpenCode 的广度确实更大，它面向更多 channel，也更像一个通用入口。Claude Code 的边界更窄，主要是 coding agent，目标是把代码任务这一件事做深做稳。正因如此，两者的比较不能只看“谁连接的场景更多”，还要看“谁在真实任务里承受过更长周期的安全、恢复、记忆、缓存和用户反馈压力”。

可是，同一段话里，讲者也给出另一组事实：OpenCode 大约由创始人在两个月内快速写出几十万行代码；Claude Code 则大概经历了一两年演进，有工程团队持续维护，并且有大量用户反馈进入商业化运转。两者的差距因此不只来自代码量，更来自反馈周期和生产环境压力。一个系统只有真的被用户反复使用，才会暴露工具漏调、上下文膨胀、记忆失准、安全边界、输出中断、权限误判这些细碎问题。

讲者还提出一个可复现实验：把相同任务交给 OpenCode 和 Claude Code，并让它们使用相同模型，例如演讲中提到的 GPT-4.7。结果在他观察中，超过 90% 的场景里，OpenCode 的效果不如 Claude Code，表现为“办事不靠谱”。这里的“同模型”很重要，它把问题从“模型聪明不聪明”移开，逼迫读者去看模型外层的工程差异。

OpenClaw vs. Claude Code：广度与深度的两个极端		
同一个模型，不同的 Harness，效果天差地别		
两种路线对比		
	OpenClaw	Claude Code
定位	通用 Agent 框架	Coding Agent
代码量	两个月几十万行	51 万行 TypeScript
功能广度	大量功能，即将能用	只做 coding，做到极致
同模型效果	基准	90%+ 场景更优
架构成熟度	概念验证级	生产级
方汉（昆仑万维创始人）测试：春节同一任务、同一模型，90%+ 情况 Claude Code 更好。方汉类比当年的中文 Linux——Linux 对社区的治理水平比 OpenClaw 创始人高很多。		
OpenClaw 是否会成为操作系统？		
OpenClaw 的贡献：重新定义了 Agent 的交互范式：		
1. 更像一个“人”持续沟通，不再有“session”的概念		
2. 所有插件通过自然语言安装和交互，无需通过 GUI 安装和配置		
3. 使用 Skills + CLI（命令行）取代 MCP（工具），实现 Agent 能力的轻松扩展，不懂技术的人也可以用自然语言编写 Skill		
但架构层面缺乏深度。例如：		
1. 缺少 Harness 深度：只有让模型能做事的上下文和工具，缺少让模型办事靠谱的错误恢复、安全机制，导致办事不靠谱		
2. 原生记忆系统效果差：原生记忆机制过于简陋，需要搭配更好的第三方记忆系统		
3. 浪费 token：KV Cache 不友好，上下文压缩机制简陋，导致 token 开销高		
4. 与多人交互时混淆身份：与多人交互时分不清是谁说的，还是陌生人说的		
5. 缺少异步通知：外部事件触发，通知系统没做成一等公民		

图 2: OpenCode 与 Claude Code 的对比页把“广度”和“深度”放在同一张表里：同一模型下，外层工程深度会显著影响任务表现。²

²视频画面时间区间：00:01:10-00:01:55。

本节的核心问题

如果同一个模型接入两个 Agent 产品，却稳定地产生不同任务表现，差异通常来自模型外围的工程系统：任务如何进入上下文，工具如何被组织，错误如何恢复，记忆如何保留，权限如何收束，成本如何被控制。本节先提出这个问题，完整定义留给下一章。

1.3 为什么“办事不靠谱”常来自 Harness 缺口

讲者没有否定 OpenCode 的价值。他明确提到，“Skills + CLI”比简单堆 MCP 工具更有价值，因为它让能力以更可组合、更贴近开发者工作流的形式暴露出来。这个判断很现实：过去一年很多 Agent 项目热衷于加 MCP，把可用工具越接越多；工具数量上去以后，模型反而变笨。原因有两层。

第一层是 token 成本。每个工具都要带描述、参数、调用约束和返回格式，工具列表一旦膨胀到几百上千个，光是让模型“看见有哪些选择”就会占掉大量上下文窗口。第二层是结构问题。工具在模型眼里常常像一个平坦列表，工具之间没有层级、依赖、互斥、优先级和场景边界。人类工程师会自然地区分“先查状态、再改配置、最后验证”；模型面对一堆平铺工具时，这种任务编排需要额外工程来补足。

工具越多，Agent 未必越强

给 Agent 加工具像给新人一整面墙的按钮。按钮多代表动作空间大，但没有分组、说明、权限、验证和恢复路径时，动作空间会变成噪声空间。模型要花更多 token 读按钮说明，还要在缺少结构的情况下猜哪个按钮该先按。

这就是讲者所谓“不靠谱”的具体形状：OpenCode 在很多架构层面缺少深度，例如错误恢复、安全机制、记忆系统、缓存友好性，以及对 token 浪费的控制。这里先不展开每一项机制；后面的章节会分别讨论错误恢复、安全、记忆、评测和数据飞轮。本节只需要抓住一个入口：生产级 Agent 的竞争，已经从“能不能调用模型”推进到“能不能把模型放进可靠工程环境里”。

因此，Claude Code 源码泄露之所以值得讲，关键不在泄露本身的刺激感，关键在它让外界看到一个成熟产品的工程厚度。OpenCode 展示了概念原型的速度和交互想象力；Claude Code 则让人看见，长期用户反馈、团队维护、商业化运行会把多少看似琐碎的细节压进产品。接下来要讨论的 Harness Engineering，正是给这些细节一个统一的工程视角。

1.4 本章小结

本章完成了三个铺垫。第一，Claude Code 源码泄露可以作为工程剖面来读，重点是注释、兜底路径、产品痕迹和真实运行压力。第二，OpenCode 与 Claude Code 的差异，在同模型任务对比中更清楚：OpenCode 有通用框架、自然语言插件配置和“Skills + CLI”的产品亮点；Claude Code 的优势来自更长周期的团队工程、大量用户反馈和商业化运行。第三，“办事不靠谱”常常来自模型外层的工程缺口，包括工具组织、token 成本、错误恢复、安全、记忆和缓存友好性。下一章将正式解释 Harness Engineering 如何把这些分散问题纳入同一个框架。

2 Harness Engineering: Agent 能力的上限与下限

2.1 从 Prompt 到 Context 再到 Harness

演讲在 00:03:57 把问题从 Claude Code 的源码细节推进到一个更一般的工程框架：Harness Engineering 到底是什么。讲者给出的第一条线索很朴素：Agent 的发展经历了三个层级，从 Prompt 到 Context，再到 Harness。Prompt 时代关心“怎么问”；Context Engineering 关心“给模型看什么、让它调用什么工具”；Harness Engineering 进一步关心“模型外部如何约束、验证、纠正，并让系统可以产品化运转”。

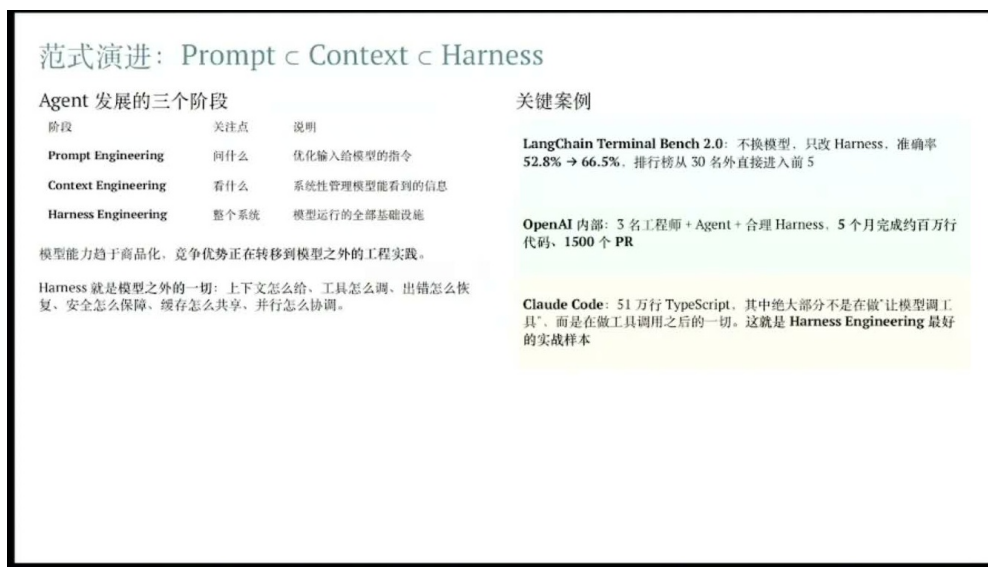


图 3: 范式演进页把 Agent 工程从 Prompt、Context 推进到 Harness，并用 benchmark 与 Claude Code 工程量说明外层系统的重要性。³

核心定义

Harness Engineering 的本节定义：Harness Engineering 是围绕模型外围建立的一套工程系统，用来组织上下文和工具，设置行动边界，验证中间结果，纠正错误路径，并把一次次模型调用变成可持续运行的 Agent 产品。它包含 prompt、上下文、工具调用、权限、记忆、压缩、旁路判断和错误处理等机制，范围大于单条 prompt，也大于一个工具列表。

这个定义故意保留“工程系统”四个字。若只说 Harness 是提示词，读者会漏掉权限分类、记忆修剪、上下文压缩这些隐藏机制；若只说 Harness 是工具，读者又会漏掉验证和纠错。更准确的理解像自动驾驶系统：模型提供感知和判断能力，Harness 像车道线、刹车控制、仪表盘、传感器融合和安全冗余，把能力约束在可执行、可恢复、可审计的轨道里。

2.2 模型、Observation space 与 Action space

讲者在 00:04:21 把 Agent 拆成三件关键物：模型、上下文加工具、以及 Harness。模型是基础智力来源，决定推理、生成、代码理解和语言表达的底座；上下文和工具决定 Agent 能看到什么、能做什么；Harness 则决定系统在真实环境中能不能稳定地做对事。

³视频画面时间区间：00:03:57–00:04:20。

用更形式化的语言，一个 Agent 在时刻 t 可以被看作从状态中读取观察，再选择行动：

$$o_t \in \mathcal{O}, \quad a_t \in \mathcal{A}, \quad S_{t+1} = F(S_t, o_t, a_t)$$

- S_t : 时刻 t 的任务状态，包括对话、文件、工具结果、缓存内容和记忆摘要。
- o_t : 时刻 t 的 observation，即 Agent 新看到的信息，例如用户输入、文件内容、命令输出或检索结果。
- a_t : 时刻 t 的 action，即 Agent 选择执行的动作，例如编辑文件、调用工具、检索记忆、生成回复。
- $\mathcal{O}$: Observation space, Agent 可观测的信息集合。
- $\mathcal{A}$: Action space, Agent 可执行的动作集合。
- F : 状态更新过程，表示一次观察和行动之后，任务状态如何变化。

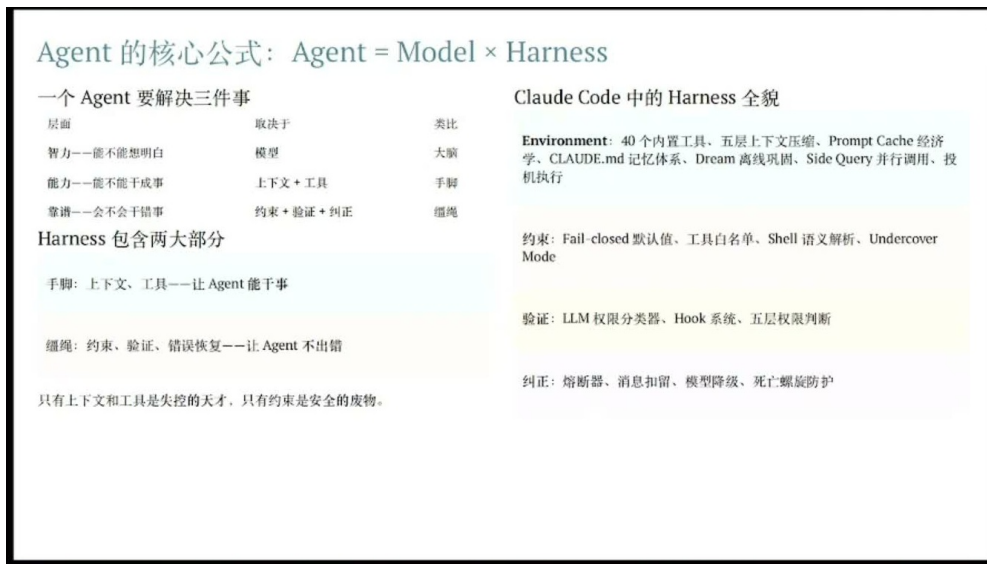


图 4: Agent 被拆成模型、上下文与工具、约束验证纠正三类要素；右侧列出 Claude Code 中可见的 Harness 机制。⁴

这组符号想说明一件具体的事：Agent 的能力上限常常受 $\mathcal{O}$ 和 $\mathcal{A}$ 限制。一个代码 Agent 如果看不到仓库文件，它再聪明也只能猜；如果看得到文件却没有写入权限，它只能提出建议；如果能写文件、跑测试、读错误日志，它的动作空间扩展，任务质量才有机会提高。能力上限来自“可见性”和“可行动性”的乘积，而这两者恰好由 Context Engineering 和工具设计托住。

直觉类比

Observation space 与 Action space 的直觉：观察空间像工程师的屏幕、日志和文档柜；动作空间像工程师手边的 IDE、终端、CI 按钮和发布权限。少了屏幕会盲修，少了按钮会空谈，按钮太多又会引入误操作风险。Harness 的任务，是在扩展可见性和行动力的同时，给危险动作加上刹车和复核。

⁴视频画面时间区间：00:04:24–00:04:56。

2.3 Harness 的三类动作：约束、验证、纠正

讲者在 00:04:43 问了一个关键问题：什么决定 Agent 的能力下限？答案落在 Harness 上，尤其是传统 Context Engineering 外围的三类动作：约束、验证和纠正。

约束负责划边界。它规定 Agent 可以读哪些目录、可以调用哪些工具、哪些操作需要用户确认、哪些内容不能外发。验证负责检查中间结果。比如代码 Agent 写完补丁后跑测试，文档 Agent 编译 PDF 后检查空白页，数据 Agent 生成结论前核对样本覆盖。纠正负责把偏离轨道的执行拉回来。比如检索结果不足时扩大查询，工具调用失败后换路径，长上下文溢出前做压缩。

能力下限

能力下限来自 Harness。模型很强只说明“有可能做对”；Harness 好才意味着系统在长任务、脏输入、工具失败、上下文膨胀和权限边界里仍有较高概率做对。生产级 Agent 的质量，常常毁在这些细部机制上。

这一点也是本节和下一章的分界。本节先讲上下文、工具和记忆如何支撑能力；安全致命要素、敏感信息外发和错误恢复会在后续章节展开。

2.4 Prompt caching、上下文压缩与 side query

00:05:13 开始，讲者用 Claude Code 的具体机制解释 Harness 为什么会影响架构决策。第一项是 Prompt caching。这里字幕里的“Prompt Catching”应按语境校正为 Prompt caching。它的作用是把重复出现的长上下文缓存起来，降低重复输入成本，也改变系统如何拆分 prompt、工具说明和任务历史。

讲者在 00:05:15–00:05:27 的措辞更重：Prompt caching 重要到会反过来牵引架构取舍。也就是说，系统提示词怎样稳定、工具 schema 怎样分层、项目背景怎样复用、历史摘要怎样更新，都要考虑缓存命中率和缓存边界。缓存如果设计差，后面的上下文压缩、工具组织和成本控制都会被拖累。

Prompt caching 的价值不只在省钱。对于长任务 Agent，系统提示词、工具 schema、项目说明和历史摘要会反复出现；缓存命中以后，设计者才敢把稳定背景放得更厚，把变化部分留给当前任务。换句话说，缓存机制会反过来塑造上下文布局。

第二项是上下文压缩。讲者提到 Claude Code 需要“五层”的上下文压缩管线，重点落在“长历史必须被加工成可继续使用的状态”。一次 Agent 任务可能包含几十轮对话、多次工具返回、若干失败路径和中间产物。若全部原样塞回窗口，token 成本会上升，注意力会被噪声稀释；若粗暴丢弃，又会丢掉关键约束。压缩的工程难点在于保留任务目标、已验证事实、未解决问题和用户偏好。

第三项是 side query，也就是主 Agent 循环之外的小模型调用。讲者在 00:05:39 强调，主 Agent 的 react loop 不能成为唯一调用 LLM 的地方。Claude Code 源码里存在许多外围小模型：有的做权限分类，有的做记忆检索，有的生成 session 标题，有的生成 todo summary。这些调用像后台小传感器，不直接承担完整任务，却持续给主循环提供判断材料。

⁵ 视频画面时间区间：00:05:13–00:05:27。

⁶ 视频画面时间区间：00:05:27–00:05:36。

⁷ 视频画面时间区间：00:05:36–00:06:20。

Prompt Cache: 第一天就要考虑的架构约束, 不是优化

如果只能从 Claude Code 源码中学一条原则, 我选这条

缓存边界写进了系统提示的物理结构

```
[全局稳定内容 - 跨用户可缓存]
--- DYNAMIC BOUNDARY ---
⚠ WARNING: Do not remove or reorder
  without updating cache logic
[会话特定内容 - 不缓存]
```

系统提示词的组织结构首先由缓存边界决定, 其次才是语义逻辑。大部分人习惯按语义分段写 prompt, 但在生产系统里, 按缓存边界分段更重要。

Fork Agent 的缓存共享

每次 fork 传入 CacheSafeParams —— 系统提示、用户上下文、工具列表、消息前缀、thinking 配置——必须和父 Agent 字节级一致才能命中一份 Cache。整个代码库 9 个 fork 调用者都走这套。

每个架构决策都在为缓存让路

设计点	缓存考量
系统提示分段	全局/动态边界分离
工具定义位置	放在系统提示稳定缓存
Fork agent 参数	CacheSafeParams 字节级一致
工具结果截断	冻结替换字符串
Thinking config	maxOutputTokens 影响 cache key
消息序列化	确定性 JSON key 顺序

Cache 命中 vs 未命中 = 成本和延迟的量级差别。它决定了消息怎么序列化、子 Agent 怎么分叉、工具结果怎么存储。第一天就画缓存边界图。

图 5: Prompt caching 相关页强调缓存边界会反过来塑造系统提示、工具定义、fork agent 参数和消息序列化。⁵

五层上下文压缩管线

不同类型的信息有完全不同的“保质期”, 需要完全不同的处理方式

Layer 1 Tool Result Budget	巨量输出存磁盘, 模型只看预览 将决策冻结以保护缓存	秒
Layer 2 HISTORY_SNIP	最精细裁剪, 纯噪声直接删掉 搜索返回 500 行但只用了 3 行, 不值得摘要	秒
Layer 3 Microcompact	在 API 缓存层面做编辑 (cache_edits) 本地消息不变, 压缩完全在 API 层完成	中
Layer 4 CONTEXT_COLLAPSE	旧对话轮次归档为摘要 保留结构——哪一轮做了什么, 结论是什么	秒
Layer 5 Autocompact	最后兜底, 先 session memory 再全量 熔断器: 连续 5 次失败放弃 (曾有会话失败 5272 次 / 全球浪费 ~250K 调用/天)	组组

核心原则: 前面能搞定就不触发后面。L1-L3 是“不丢信息的压缩”——数据还在磁盘上, 只是模型不看全文。L4-L5 是“有损压缩”——信息被摘要替代。大部分时候最重的 Autocompact 根本不需要跑。

图 6: 五层上下文压缩管线展示从工具结果预算、历史片段、微压缩到会话级 Autocompact 的分层处理。⁶

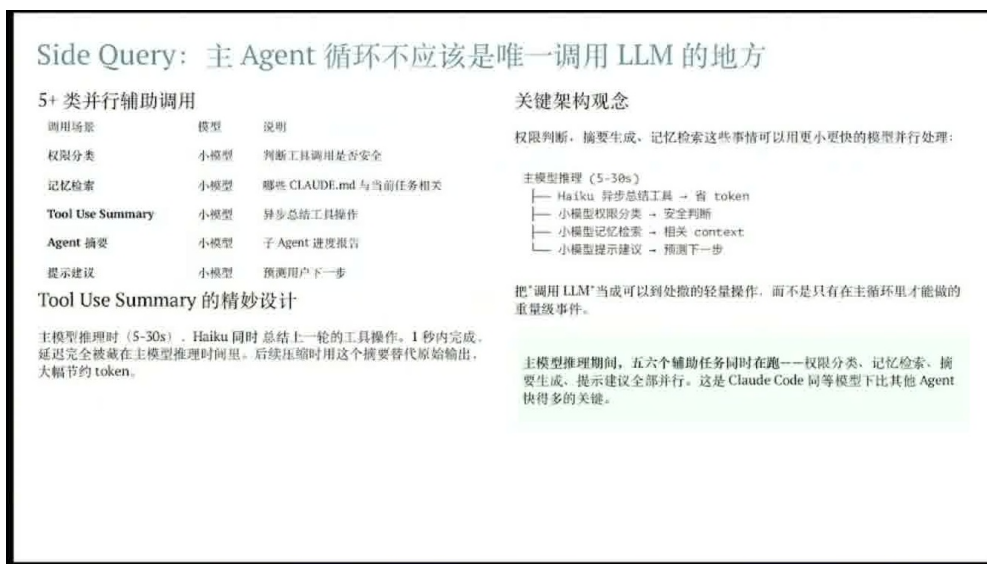


图 7: Side query 页把权限分类、记忆检索、工具使用摘要、Agent 摘要和提示建议拆成并行辅助调用，说明主循环之外还存在轻量判断链路。⁷

工程意义

Side query 的工程意义：主循环适合做目标分解和行动决策；旁路查询适合做窄任务判断，例如“这个命令危险吗”“这段历史该不该召回”“这个会话标题叫什么”。把窄任务拆给小模型，可以降低主循环负担，也让权限、记忆和摘要这些机制更容易被单独评测。

常见误解

常见误解：把 Agent 想成“一个大模型反复调用工具”会低估 Harness 的复杂度。真实产品里常有多个辅助判断链路，主模型输出只是可见部分；缓存、压缩、旁路分类和记忆检索这些后台链路，常常决定体验是否稳定。

2.5 Markdown 文件系统记忆、vector database 的边界与 dream 机制

00:06:20 之后，讲者转向记忆架构。他观察到 Claude Code 和 OpenCode 都采用了 Markdown 加文件系统的记忆方式。这看起来原始，却很适合通用 Agent：Markdown 兼具人类可读性、层级结构、标题索引、列表和代码块；文件系统天然提供路径、命名、增量编辑和版本管理。对于需要被人类审查和长期维护的记忆，它比一堆不可见向量更容易治理。

这里要补上一层背景：向量数据库擅长相似度检索，常见模式是给查询 q ，从数据集 D 中取最相似的 Top- K 条结果：

$$R_K(q, D) = \text{TopK}_{x \in D} \text{sim}(q, x)$$

- q ：当前查询，例如“用户过去对代码风格有什么偏好”。
- D ：可检索的数据集合，例如历史对话片段或笔记片段。
- K ：返回结果数量，例如 Top-10 中的 10。
- x ：数据集合中的一个候选片段。

- $\text{sim}(q, x)$: 查询和候选片段之间的相似度分数。
- $R_K(q, D)$: 最终返回的 Top-K 检索结果集合。

讲者用 00:06:47 到 00:07:16 的猫例子说明 Top-K 的覆盖问题：如果有 100 个猫样本，其中 90 只是黑猫、10 只是白猫，检索 Top-10 很可能全是黑猫。此时系统会误以为样本里没有白猫。再问“这里一共有多少只猫”，Top-K 检索也很难给出总数，因为它每次只看最相似的一小撮片段，天然缺少全量覆盖和聚合能力。

检索边界

向量数据库的边界：向量检索适合“找相似片段”，不擅长“保证覆盖少数类”“统计全量数量”“维护长期结构化知识”。把原始记忆全塞进向量库，Agent 会在局部相似性里显得聪明，在覆盖性、计数和长期一致性上暴露短板。

所以讲者强调，原始数据需要额外过程来结构化整理、压缩总结。Markdown 正好承担“知识表达”的角色：标题把主题切开，列表保存约束，段落记录判断，代码块保存可执行片段。知识图谱在特定专业领域可以更精确，但通用 Agent 面对的材料很杂，Markdown 的弹性和可维护性更高。

记忆架构：为什么用 Markdown + 文件系统这种最原始的方法

向量数据库的根本问题：Top-K 检索不完整

一个简单的例子：系统见过 100 只猫。其中 90 只黑猫 + 10 只白猫。用向量数据库取 Top-10——

- ❶ 决策错误：Top-10 很可能全是黑猫，一只白猫都没有。基于 Top-K 的样本做判断，结论必然偏颇
- ❷ 无法计数：如果想统计“黑猫 vs 白猫”的比例，向量数据库根本做不到——它没有办法把原始 100 个例子全部匹配出来

不是说向量数据库不好，是说单靠它存原始数据不够。必须对原始数据做结构化整理、压缩、总结——这正是 Markdown 的作用。

为什么选 Markdown 而不是知识图谱

- 自然语言是 LLM 最擅长的格式——Markdown = 结构化的自然语言
- 知识图谱在专业领域更精确，但在通用场景下，自然语言的灵活性和覆盖面更强
- 可编辑可审计：打开就能看 AI 记了什么，记错了直接改；Git 可追溯

Claude Code 的 Markdown 记忆体系

文件	用途
CLAUDE.md	项目级记忆与约定
MEMORY.md	核心事实与用户偏好
memory/YYYY-MM-DD.md	按日期归档的交互日志

Dream：睡眠学习机制

人类睡眠时大脑巩固记忆——Agent 空闲时巩固会话记忆：

- 扫描近期会话，提取值得长期保留的信号
- 合并到已有主题文件，删除被推翻的旧事实
- 修剪索引，保持精简

Dream 做的事，向量数据库做不了：它需要理解“这条信息是不是推翻了旧的”、“这 90 只黑猫能不能总结成一条规律”——这些都需要自然语言层面的推理，不是向量相似度能解决的。

The Bitter Lesson：最终胜出的方法一定是能利用更多计算资源的通用方法。Markdown + 自然语言组织 + 文件系统，再叠加 LLM 做压缩、整理，这是通用、可扩展的路径。

图 8: 记忆架构页同时呈现 Top-K 检索覆盖缺口、Markdown 文件记忆体系与 Dream 睡眠学习机制，适合作为本节记忆讨论的总图。⁸

最后是 dream，也就是睡眠学习机制。讲者在 00:07:54 提到 Claude Code 里有一个叫 dream 的机制，会扫描近期对话，定期修剪、整理和总结历史记忆。这个命名很形象：人在睡眠中会把白天经历重组为更稳定的记忆；Agent 的 dream 机制也在后台把散乱的交互历史变成更短、更结构化、更可复用的知识。

记忆维护

记忆重在维护。长期 Agent 需要的不止是“把所有历史留下来”，还要周期性地清理重复、压缩结论、保留例外、更新偏好，并让人类可以检查这些记忆。Markdown 文件系统记忆、上下文压缩和 dream 机制共同构成了这条维护链路。

⁸视频画面时间区间：00:06:47–00:08:08。

2.6 本章小结

本章完成了 Harness Engineering 的第一组定义。Agent 可以理解为由模型、上下文、工具和 Harness 共同形成的可行动系统；模型提供基础智力，Observation space 决定它能看到什么，Action space 决定它能做什么，Harness 决定它在真实任务里能不能稳定、可控、可恢复地运行。

从 00:05:13 到 00:08:08 的 Claude Code 案例给出了一组具体机制：Prompt caching 改变上下文布局 and 成本结构；上下文压缩把长任务历史变成可继续使用的状态；side query 把权限分类、记忆检索、标题生成和 todo summary 交给外围小模型；Markdown 文件系统记忆让长期知识可读、可编辑、可治理；向量数据库适合相似度检索，但在覆盖、计数和长期结构化上有天然缺口；dream 机制负责周期性整理近期对话，把历史压缩成更可用的记忆。

这一章的要点很硬：Agent 工程的关键差距，常常藏在模型调用之外。看得见、做得到，只是上限；约束、验证、纠正和记忆维护，才托住下限。

3 生产级 Agent 的安全、权限与错误恢复

上一节已经把 Harness Engineering 定位为模型外围的约束、验证、纠正和产品化运转系统。本节不重新定义 Harness，只把镜头推进到其中最硬的一层：当 Agent 已经能读文件、看邮件、调工具、改代码、发请求时，怎样让它少犯错、少泄漏、少把半成品动作当成完成。演讲在 00:08:08–00:08:22 处给出转场：前面讲的上下文、工具和记忆，是提高 Agent 能力上限；接下来讨论的安全、权限和恢复，是降低出错概率的底盘。

这个转场很关键。对普通聊天机器人来说，错误常常停留在“答错一句话”；对生产级 Agent 来说，错误可能变成读入 API key、采信恶意邮件、向外发送秘密、执行危险命令、任务中断后留下不一致状态。也就是说，Agent 的风险来自模型输出文本，也来自它连接到真实世界后的动作后果。

这里还要补上讲者在 00:08:30–00:08:42 的一个关键 caveat：Agent 的安全机制要在初始架构里进入设计，后期再补会非常困难。原因很直接：一旦系统已经默认允许模型读文件、看邮件、调工具、执行命令和外发内容，权限边界就会散落在工具协议、prompt、插件、缓存、日志和用户习惯里。后补安全等于在已经通车的城市里重铺红绿灯和隔离带，成本高，遗漏多，也容易破坏原有体验。讲者把 OpenCode 的安全问题归因到这一点：问题来自某个单独检查缺失，也来自安全边界没有在架构早期形成统一约束。

3.1 Agent 致命三要素

演讲在 00:08:42–00:09:08 提出“致命三要素”：第一，Agent 能访问用户的敏感信息；第二，Agent 会暴露于不受信任的内容，例如外部邮件、网页、Issue、PR 评论；第三，Agent 能自主执行敏感或危险操作。单看任何一项都未必致命，三项叠加后，风险会从“模型理解错了”升级成“系统替攻击者完成了闭环”。

一个直观例子出现在 00:09:10–00:09:22：Agent 可以看到用户的 API key，同时收到一封外部邮件，邮件里指示它把 API key 交出来；如果 Agent 缺少权限判断和外发审查，就可能把 API key 通过邮件发出。这里的问题不只是一条 prompt injection。完整攻击链包含三个节点：秘密可见、恶意指令可见、外发动作可用。

⁹视频来源区间：00:08:24–00:10:05。

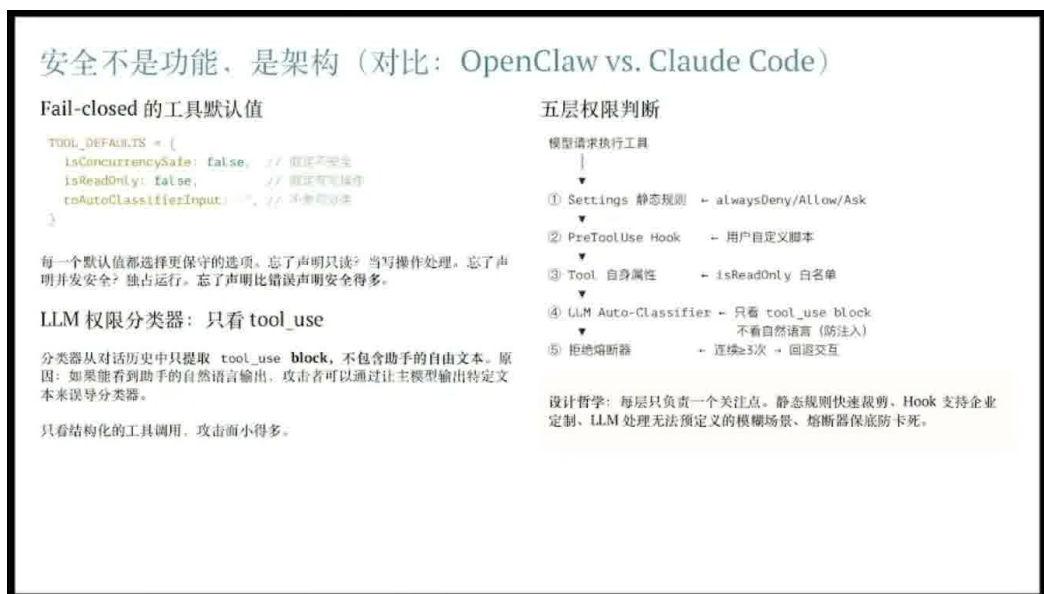


图 9: 安全架构页把 fail-closed 默认值、LLM 权限分类器和五层权限判断放在同一张图里：Settings 静态规则、PreToolUse Hook、工具自身属性、LLM Auto-Classifier 与拒绝解析器共同形成读密钥、调工具、外发内容之前的多道门禁。⁹

风险闭环

致命三要素可以压缩成一个二值风险门：

$$F = I_{\text{secret}} \cdot I_{\text{untrusted}} \cdot I_{\text{action}}$$

其中：

- I_{secret} ：当前状态中是否可接触敏感信息，例如 API key、token、私有代码、客户数据。
- $I_{\text{untrusted}}$ ：当前上下文中是否混入不可信内容，例如外部邮件、网页、第三方 issue、用户上传文件。
- I_{action} ：Agent 是否拥有自主执行敏感动作的能力，例如发邮件、提交代码、运行命令、调用外部 API。
- $F = 1$ ：三项同时成立，系统进入高危区；此时默认策略应从自动执行切换为确认、降权或拒绝。

这个公式故意简单。它像电路里的“三个开关串联”：任意一个开关断开，攻击链就难以闭合；三个开关都闭合，系统不该继续依赖模型的“自觉”。生产级安全设计的目标，就是在读取、理解、执行、外发的每一步插入可审计的开关。

3.2 敏感信息读取与外发的双向防护

演讲在 00:09:22-00:10:11 说明，Claude Code 一类系统会在工具调用上做多层校验。用户要求读取 API key 时，系统不会把内容简单塞进上下文；除非使用类似 `dangerously_skip_permissions` 的特殊选项，否则会询问用户是否允许读取，并用小模型和规则共同判断这次读取是否危险。外发

方向也需要同等严肃：让 Agent 发 email、提交表单、调用 webhook 时，系统要审查消息体里是否携带敏感信息。

这里有一个容易被忽略的工程事实：输入侧防护和输出侧防护解决的是两个不同问题。输入侧防护关注“秘密是否进入模型上下文”；输出侧防护关注“秘密是否离开可信边界”。如果只拦输入，Agent 仍可能把历史上下文、工具返回、文件片段中的秘密带出；如果只拦输出，秘密已经被模型读入，后续推理、摘要、缓存、日志都可能扩大暴露面。

权限分级

权限分类可以写成一个更实用的分层函数。设 a_t 是 Agent 在时刻 t 准备执行的动作， S_t 是当前任务状态，则风险分数可以近似表示为：

$$R(a_t, S_t) = w_s r_s + w_u r_u + w_e r_e + w_p r_p + w_c r_c$$

其中：

- r_s ：动作是否读取或触碰敏感数据。
- r_u ：决策依据中是否包含不可信内容。
- r_e ：动作是否把内容发送到外部边界，例如 email、网络请求、公开仓库。
- r_p ：动作是否会产生持久副作用，例如改文件、提交代码、删除资源、修改权限。
- r_c ：命令语义是否复杂或容易伪装，例如脚本拼接、管道、编码后的参数。
- w_s, w_u, w_e, w_p, w_c ：不同风险维度的权重，由产品场景决定。

再把 R 映射为权限等级：

$$L(a_t, S_t) = \begin{cases} \text{allow}, & R < \tau_1 \\ \text{ask}, & \tau_1 \leq R < \tau_2 \\ \text{deny}, & R \geq \tau_2 \end{cases}$$

$L = \text{allow}$ 表示可自动执行； $L = \text{ask}$ 表示需要用户确认或展示 diff； $L = \text{deny}$ 表示拒绝执行或要求降权重写计划。这个形式化属于本节的工程抽象，它忠实表达了 00:09:22–00:10:11 的机制：用规则和小模型把“能不能读、能不能发、能不能执行”变成显式门禁。

从用户体验看，确认弹窗看起来像小细节；从系统安全看，它是把不可逆动作改成可审计动作。真正危险的 Agent 产品，常常在 demo 中显得更“丝滑”，因为它少问确认、少做审查、少暴露中间判断。生产级系统要承认一个事实：必要安全摩擦是产品责任，缺少必要摩擦才是坏味道。

3.3 错误恢复：从工具漏调到输出中断

00:10:12–00:11:12 的重点从安全转到错误恢复。演讲者提到，Claude Code 源码泄露后，他很快把其中大量错误恢复机制迁移到自己的 Agent 里，因为这些问题在真实产品中反复出现：模型该调工具时没有调，任务没做完就结束；供应商在输出中途卡死；输出撞到 8k token 上限；内容生成到一半中断后，系统不知道能否从上一个位置续写、继续思考或重新规划。

这类故障和“模型答错”不同。它们更像分布式系统里的失败：网络会断、服务会超时、事务会半提交、日志会缺尾巴。生产级 Agent 要把每一次工具调用、每一段输出、每一个中间状态都当

成可恢复对象，而非一次性对话流。

错误恢复：在确认无法恢复之前，不暴露中间态

输出触顶的两步恢复

第一步：静默升级上限——撞 `max_output_tokens` 时。如果是默认 8K，直接用 64K 重发。对用户完全透明，只发一次。

第二步：多轮续接（最多 3 次）——64K 还不够，才注入 meta 消息：“Resume directly — no apology, no recap. Pick up mid-thought.”

消息扣留（Message Withholding）

恢复期间，错误消息被扣留，不发给用户（桌面端、IDE 插件）。因为它们看到 `error` 就会终止会话。

恢复成功 → 消费者永远不知道出过错
全部失败 → 释放扣留的错误

模型降级

主模型不可用时，自动剥离旧模型的签名块和 `tool_use` 格式，用备用模型重发。

死亡螺旋防护

API 出错 → 触发 `stop hooks` → `hooks` 也调 API → 也出错 → 又触发 `hooks`.....

规则：API 错误时跳过所有 `stop hooks`。宁可丢失一次记忆提取，也不能让系统陷入无限循环。

熔断器无处不在

场景	阈值	真实数据
上下文压缩	连续 3 次	曾浪费 ~250K 调用/天
权限分类	连续 3 / 累计 20	防 Agent 被卡死
输出触顶	最多 3 次续接	先升级再续写

这些阈值不是拍脑袋定的——3 次阈值来自 1279 个会话曾出现 50+ 次连续失败的真实产线数据。

错误处理的边界不是“单次 API 请求”，而是整个恢复循环。

图 10: 错误恢复页展示生产级 Agent 需要处理的恢复场景：输出触顶后的两步续写、恢复期间的消息扣留、主模型不可用时的模型降级、stop hook 死亡螺旋防护，以及熔断器阈值对上下文压缩、权限分类和输出续写的保护。¹⁰

恢复风险

错误恢复最怕“看起来完成”。例如 Agent 没有调用必要工具，却用自然语言宣布任务结束；或者输出到一半撞到 token 上限，前半段文本看起来流畅，后半段结构已经丢失。生产系统需要记录任务检查点：计划、已调用工具、工具返回、已写入文件、剩余 TODO、最后一个可靠输出位置。没有检查点，所谓“继续”只是重新猜。

可恢复的 Agent 循环至少要包含四个动作。第一，检测异常：工具未调用、超时、空返回、输出截断、供应商错误码。第二，保存状态：把 S_t 中和任务进展有关的内容压成结构化 checkpoint。第三，选择恢复策略：重试工具、换供应商、缩短上下文、要求用户确认、从断点续写。第四，验证完成：不能只看模型说“完成了”，还要看文件、命令返回、测试结果或外部系统状态。

纠错层

错误恢复是 Harness 的纠错层在生产环境里的具体形态。它不追求让模型永远不犯错；它追求错误发生后，系统能发现、能定位、能继续、能回滚到可理解状态。对长任务 Agent 来说，这一层常常决定产品能不能从玩具变成工具。

3.4 Sub-agent 工具集合与角色边界

演讲在 00:11:13-00:11:40 回到 Agent 安全：需要限制 Agent 能执行的工具集合。Claude Code 内部有多个 sub-agent，不同 sub-agent 的工具集合不同；工具集合定义了角色，也定义了它能做的事情。

¹⁰ 视频来源区间：00:10:14-00:11:05。

这句话的工程含义很直接：角色不能只写在 prompt 里。一个“只负责搜索”的 sub-agent 如果仍然能写文件、发邮件、运行 shell，它的角色边界就是纸面边界。真正的边界要落到工具集合 $\mathcal{T}_i$ 上。设第 i 个 sub-agent 的可用工具集合为 $\mathcal{T}_i$ ，它能采取的动作空间就是：

$$\mathcal{A}_i = \{a \mid \text{tool}(a) \in \mathcal{T}_i\}$$

其中：

- $\mathcal{T}_i$ ：第 i 个 sub-agent 被授权使用的工具集合。
- $\mathcal{A}_i$ ：该 sub-agent 实际可执行的动作空间。
- $\text{tool}(a)$ ：动作 a 所依赖的具体工具，例如读取文件、写文件、运行命令、发邮件。

这个集合约束像办公室里的门禁卡：文案人员可以进文档库，财务人员可以进付款系统，实习生可以看只读面板。口头上说“请不要进机房”不够，门禁卡要真的打不开机房门。Agent 也是一样，sub-agent 的可靠性来自工具级授权，而非角色描述的礼貌措辞。

Agent 安全：限制工具集合、基于语义的命令行安全检查

Agent 类型	工具面
主 Agent	完整工具集
子 Agent	禁止递归创建 Agent，退出计划模式
Coordinator	只有 5 个工具：创建/发消息/停止 worker
Async Agent	只有文件读写、搜索、shell
In-process Teammate	Async = 任务管理 + cron

Capability-based 的身份定义比 system prompt 角色提示词更硬，更可审计，更难绕过。

Shell 安全：语义解析 > 关键词黑名单

完整的命令语义解析器，每条 git 子命令的每个 flag 做类型化标注。

真实安全案例：

```
git diff -S -- --output=/tmp/pwned
```

- S 之前被标记为 none（不带参数）
- 但 git 的实际行为是：-S 强制消费下一个 argv
- 攻击路径：验证器认为 -S 不带参数 → 推进 1 个 token → 遇到 -- 停止检查 → --output 未检查 → 任意文件写入
- 修复：把 -S 改成 string 类型

普遍规律：安全机制的粒度应该和攻击面的粒度匹配。关键词黑名单粒度太粗，语义解析才能覆盖攻击面。

图 11: 工具集合页把角色边界和命令安全连在一起：主 Agent、子 Agent、Coordinator、Async Agent 与 in-process Teammate 拥有不同工具集；Shell 安全侧强调语义解析优先于关键词黑名单，因为攻击面来自命令真实意图、参数类型和数据流向。¹¹

3.5 语义命令解析优于关键词名单

00:11:40–00:12:01 进一步指出，命令行安全检查应基于语义解析，防护效果明显强于关键词和名单。关键词名单的问题在于它只看表面字符串。危险意图可以藏在管道、脚本、参数展开、编码文本、间接调用里；相反，某些包含敏感词的命令也可能只是只读检查或日志脱敏。

语义命令解析关注三个问题：这条命令最终会读什么、写什么、发到哪里。比如同样出现 token，一条命令可能是在本地扫描并替换成 [REDACTED]，另一条命令可能是在把环境变量打包上传。关键词视角会把二者混在一起；语义视角会追踪数据流和副作用。

¹¹ 视频来源区间：00:11:12–00:11:58。

命令解析

关键词名单适合做第一层粗筛，不能充当最终权限系统。真实命令会有别名、脚本、管道、重定向、压缩包、base64、子进程和网络端点。只盯关键词，就像只检查快递箱外面的标签，箱子里面装什么仍然未知。

更稳的做法是把命令解析为结构化意图：源、目标、数据类型、副作用、可逆性、外部边界。随后再把这些特征送入前面的风险函数 $R(a_t, S_t)$ 。这样，权限系统审查的是“把什么数据带到什么边界、造成什么持久变化”，而非某个词是否出现在命令字符串里。

3.6 本章小结

本节把第 2 节的 Harness 概念具体化为安全、权限和恢复三组机制。00:08:08–00:12:09 这一段的核心判断很硬：生产级 Agent 的难点不只在让模型更聪明，还在于让系统在看见秘密、接触外部内容、拥有真实动作能力时仍然可控。

致命三要素给出安全底线：敏感信息、不可信内容、自主敏感动作一旦闭环，就必须触发权限审查。读取 API key 和发送 email 是两个方向的边界，前者防止秘密进入上下文，后者防止秘密离开可信域。错误恢复则处理另一类生产现实：工具漏调、供应商卡死、token 上限、输出中断和断点续写。最后，sub-agent 的工具集合把角色边界落到可执行动作上，语义命令解析把权限判断从关键词表面推进到真实意图和副作用。

这节为下一节做了铺垫：当安全门禁、恢复机制和工具边界已经存在，产品团队还需要用研究式方法持续验证它们是否真的有效。也就是演讲在 00:12:09 开始转向的主题：用 evaluation、ablation 和实验系统做产品。

4 用研究方法做产品：Evaluation、Ablation 与反蒸馏

上一章讨论的是安全、权限和错误恢复，这一章把视角转到“怎么让 Agent 产品持续变好”。讲者在 00:12:07 明确给出一句概括：用做研究的方法去做产品。他也补了一句，这部分未必全部算 Harness 的组成部分；可是对生产级 Agent 来说，评测、实验和反蒸馏会直接决定外层工程能不能进化。没有评测体系，prompt 只能靠感觉改；没有 ablation，工程师分不清哪项机制真有贡献；没有反蒸馏，模型公司的行为链路和训练价值会被 API 调用者持续吸走。

4.1 Frontier lab 为什么把产品当实验系统做

讲者提到，他和硅谷 OpenAI、Anthropic、Google 等做应用的同事交流后，得到一个很稳定的观察：这些 Frontier lab 内部本来就有大量做实验的人，做产品时也沿用实验文化。这里的“实验”超出论文表格和按钮颜色测试，直接把用户任务、prompt、工具策略、记忆策略、故障恢复策略都放进可测量、可回滚、可比较的系统里。

00:12:35 到 00:12:47 这段给出了本章的第一个判断：优秀 Agent 公司与较弱 Agent 公司的很大差别，在 evaluation 体系的搭建。这个判断很硬。Agent 产品的失败常常呈现为“十次里有两三次掉链子”：该调用工具时没有调用，长任务执行到一半丢状态，摘要把关键约束压没了，恢复机制把错误继续放大。人工试用几次很容易漏掉这些边角；evaluation system 的作用，就是把这些边角变成固定样本、固定指标和固定回归检查。

Evaluation system 的产品含义

本节沿用契约中的定义：evaluation system 是以测试用例和指标驱动 prompt、策略和架构选择的实验基座。它让“这个 Agent 有没有变好”从主观印象变成可重复测量的问题。对 Agent 产品来说，评测用例通常要覆盖真实任务、工具调用、长上下文、错误恢复、权限判断和输出质量，而不能只测一句问答是否好看。

讲者举了 Manus 的例子：在他看来，Manus 是 evaluation 体系非常完善的典型公司；早在上一年同期，内部就已经有完善测试用例，并且 prompt 有一套成熟迭代系统，可以每天发布几十版不同 prompt。这个数字背后真正重要的是流水线：每个版本进入测试集，跑出指标，和上一版比较，再决定是否进入更大范围实验。



图 12: 章节过渡页把本章主题钉在“用研究的方法做产品”上，为后续 evaluation、ablation 和实验工程提供入口。¹²

4.2 Evaluation 体系与 prompt 快速迭代

一个可用的 evaluation suite 可以粗略写成一组任务样本：

$$E = \{(x_i, r_i, m_i)\}_{i=1}^n$$

- E ：评测用例集合，也就是本章的 evaluation suite。
- x_i ：第 i 个输入任务，例如“修复一个测试失败”“整理一个长上下文项目”“在权限受限时拒绝危险命令”。
- r_i ：该任务的参考结果或验收规则，可以是单元测试通过、文件 diff 正确、权限决策正确，也可以是人工标注的质量标准。
- m_i ：度量函数，负责把 Agent 的执行结果转成分数、通过/失败标签或错误类型。
- n ：评测集中任务样本的数量。

¹²视频来源区间：00:12:28-00:13:10。

这个形式化写法的价值，在于它提醒读者：Agent 的评测不该只有一个总分。代码 Agent 至少要分开看几类能力：有没有理解任务，工具调用顺序是否合理，是否遵守权限，是否能在失败后恢复，最终产物是否通过验证。一个 prompt 新版本如果让最终答案更流畅，却让敏感操作确认率下降，把它判成“整体变好”就是指标结构太粗造成的错觉。

读者容易漏掉的点：prompt 也是可发布的软件

很多人把 prompt 当成一句文案，改完就上线。成熟 Agent 团队会把 prompt 当成版本化组件：有变更记录、有测试集、有回滚路径、有灰度发布。它和普通代码的区别在于行为更随机、更受上下文影响，所以更需要评测体系托底。

从产品工程角度看，prompt 快速迭代通常包含四步。第一步，收集失败样本，例如用户任务失败、工具调用缺失、摘要误删约束。第二步，把失败样本整理进评测集，避免同类错误下次悄悄复发。第三步，生成或手写候选 prompt 版本，例如改变系统指令、工具选择说明、恢复策略说明。第四步，批量跑评测，只有指标收益明确、回归风险可接受的版本，才进入线上实验。

4.3 Ablation baseline、GrowthBook 与工程化实验

讲者在 00:13:15 开始把话题落到 Claude Code 这类产品内部的实验机制：源码里有一些 ablation baseline 的 flag。这里的 flag 可以理解成工程开关。上下文压缩、记忆整理总结、故障恢复策略，都可能有多种竞争方案；开关允许团队在同一套产品框架里关闭某项机制，或者切换到某个候选版本，再观察指标变化。

消融实验的核心问题是：“这项机制到底贡献了多少？”如果某个 Agent 同时升级了 prompt、上下文压缩、记忆整理和错误恢复，然后成功率从 70% 变成 78%，团队并不知道收益来自哪里。消融基线会把其中一项机制拿掉或换回旧版本，计算相对差值：

$$\Delta_j = \text{Score}(H_j, E) - \text{Score}(H_0, E)$$

- Δ_j ：第 j 个机制或策略相对基线带来的实验收益差值。
- H_j ：打开第 j 个候选机制后的 Harness 策略组合，例如新版上下文压缩或新版故障恢复。
- H_0 ：基线策略，通常是旧版本、关闭某个机制后的版本，或当前线上稳定版本。
- E ：固定评测用例集合。
- $\text{Score}(\cdot)$ ：评测函数，可以是任务成功率、工具调用正确率、恢复成功率、成本、延迟等指标的组合。

A/B 实验与 ablation 的分工

A/B 实验关心“两个产品版本在真实流量里谁表现更好”，典型做法是把用户或会话按随机规则分到 A 组和 B 组，再比较成功率、留存、时延、成本、投诉率等线上指标。Ablation 关心“某个内部机制有没有贡献”，典型做法是通过 flag 关闭或替换单项机制，并尽量固定其他条件。前者更像在真实道路上试两套驾驶方案，后者更像在实验台上单独拔掉一个零件，看整车性能掉多少。

两者在 Agent 产品里经常配合使用。离线 evaluation 先筛掉明显退化的策略；ablation 告诉团队上下文压缩、记忆整理、错误恢复等机制各自的贡献；A/B 实验再把少数候选版本放到真实用户

场景中，检查离线指标没有覆盖到的行为，例如用户是否更愿意继续交互、任务成本是否可接受、长任务完成率是否真的提升。

讲者提到 Claude Code、Gemini 等内部有复杂 A/B 测试系统，并点名 GrowthBook 作为实验工程基础设施。这里不需要扩展到未证实的实现细节；可以把 GrowthBook 理解为一类实验系统：它管理实验开关、用户分组、指标采集和结果分析。对 Agent 团队来说，GrowthBook 的意义在于把“选择哪个策略”变成工程流程，减少会议里的口味投票。

消融实验：把 ML 的研究方法搬到产品工程

ABLATION_BASELINE：一次性关掉 7 个功能

源码里有一个 ABLATION_BASELINE flag，注释标注为“Harness-science L0 ablation baseline”。启用后一次性关掉：

- Thinking mode
- 上下文压缩
- 自动记忆
- 后台任务
- 简单模式
- 交织思考
- 第二个后台任务 flag

跑对照实验——关掉某个功能后，任务完成率，token 消耗，会话时长有没有显著变化。

做过 ML 研究的人都熟悉消融实验。把这个方法论搬到产品工程上，在工业代码里是少见的。最接近的类比是字节跳动——几乎所有产品决策都经过 A/B 验证。

双层 Feature Flag

层级	机制	用途
编译时	Bun feature()	false 分支从二进制物理删除——安全研究员反编译也找不到
运行时	GrowthBook	灰度发布 / A/B / kill switch 三合一

GrowthBook 实验工程

- 服务端分桶 (remoteEval: true，客户端拿计算好的结果)
- 用户画像定向 (按 deviceId / organizationUUID / subscriptionType / platform / email)
- 曝光追踪 (每个 feature 的 experimentId + variationId 记到事件系统)

核心理念：不是“感觉有用就上”，是“数据证明有用才上”。压缩熔断器的 3 次阈值，autocompact 的触发条件，全部有真实生产数据支撑。

图 13: 消融实验页把机器学习研究方法迁移到产品工程：ABLATION_BASELINE 一次性关闭多项能力，双层 Feature Flag 区分编译期和运行期控制，GrowthBook 则承担实验分桶、用户画像定向与曝光追踪，让机制贡献从主观判断进入数据验证。¹³

实验工程的真正产物

实验工程产出超出“某个版本赢了”这条结论，更重要的是一张机制地图：哪些 prompt 改动稳定有效，哪些记忆策略只在长任务里有效，哪些故障恢复会增加成本，哪些权限策略会牺牲完成率。没有这张地图，Agent 团队会在复杂系统里盲飞。

4.4 Anti-distillation: fake tools、训练集投毒与 CoT 摘要

00:13:54 之后，讲者转向另一个产品级问题：模型公司非常在意自己的模型被蒸馏。这里的“蒸馏”可以用师生关系理解：强模型像老师，弱模型或外部系统像学生；学生通过大量调用老师的 API，记录输入、输出、工具调用链和推理痕迹，再训练出一个行为相似的模型。蒸馏不一定需要拿到权重，只要拿到足够多高质量行为样本，就可能复制一部分能力。

反蒸馏的目标，是降低外部调用者从 API 响应中学习到完整行为链路的价值。讲者在 Claude Code 相关源码观察里提到两层设计。第一层是 fake tools：API 后端可以在响应中注入虚假的工具调用。Claude Code 自己执行时不会受影响，因为它知道哪些调用是真正需要执行的，或者能在内部协议里识别有效调用；外部抓取者如果直接拿 API 返回链路做训练，就可能把假的工具调用学进去，训练集被污染。

¹³视频来源区间：00:13:11–00:13:54。

训练集投毒在这里是什么意思

训练集投毒并非随便制造垃圾数据，它向潜在蒸馏者可见的数据流里放入“对原系统无害、对模仿者有害”的样本。类比成防伪水印：正版系统能识别水印，盗版学习者把水印当成正文抄走，之后行为会偏离真实任务需求。

这个机制的关键在于不伤害自家产品。假工具调用如果真的被 Claude Code 当成真实工具执行，就会破坏任务；所以 fake tools 必须存在于“外部可见、内部可过滤”的层级。它像给外部观察者摆了几条看似合理的岔路，而内部执行器手里有地图，不会走错。这个设计很像安全工程里的蜜罐，也像机器学习数据集里的水印样本：它不追求完全阻止观察，只追求让批量复制的成本和风险上升。

反蒸馏：两层纵深防御

源码中独立的工程子系统——阻止竞争对手通过 API 输出训练自己的模型

Layer 1: Fake Tools 训练集投毒

```
// Anti-distillation: avoid fake tools
// opt-in for IP cli only
anti_distillation: [ 'fake_tools' ]
```

API 后端在响应中注入虚假工具调用，混在真实输出中。批量抓取训练数据的系统会一并吃进去——相当于在训练集中投毒。

攻击者两难：要么花成本过滤（但很难区分真假工具调用），要么接受训练数据被污染。

Layer 2: Connector Text Summarization

服务端把模型在工具调用之间生成的自由文本（推理过程）缓存起来，替换为摘要 + 加密签名。客户端发回签名摘要，服务端凭签名还原原文。

外部观察者只能看到摘要——模型的推理链条被签名遮蔽，只有 Anthropic 服务端能解密。

两层互补的设计逻辑

Layer 1 针对“大网捞鱼”式的批量抓取——成本低，覆盖广，靠噪声比让训练数据不可用

Layer 2 针对精细逆向工程——即使攻击者过滤掉虚假工具调用，模型推理过程仍被签名遮蔽

反蒸馏的存在本身就证明模型能力还没真正商品化。如果模型已经可替换，为什么要大力防蒸馏？

图 14: 反蒸馏页给出两层纵深防御：第一层通过 fake tools 向外部抓取者可见的数据流注入低成本干扰，第二层用 Connector Text Summarization 缓存摘要并配合签名链路，降低完整工具调用轨迹被批量学习的价值。¹⁴

第二层是思维链的可见性控制。讲者提到，Claude 当时的思维链似乎仍较透明，还能看到；OpenAI、Gemini 等系统的思维链已经做了摘要。CoT，即 Chain of Thought，指模型内部或外显的逐步推理文本。完整 CoT 对蒸馏很有价值，因为它不仅给出答案，还给出中间路线：如何分解问题、何时调用工具、如何修正错误、如何解释约束。摘要后的 CoT 仍能帮助用户理解大致过程，但可用于训练模仿者的细粒度信号会少很多。

不要把 CoT 摘要只理解成“隐藏推理”

CoT 摘要要有三层产品含义：第一，降低用户被冗长推理淹没的成本；第二，减少泄露内部策略、工具协议和安全判断的风险；第三，降低外部蒸馏者拿到高密度训练信号的价值。对模型公司来说，这同时是体验设计、安全设计和商业防御。

讲者还提到加密签名一类方向，并谨慎说明可能尚未实施。它的直觉很好理解：如果工具调用链带有内部签名，客户端或服务端就能验证“这条调用是否属于可信链路”。签名机制和 fake tools 是互补关系：fake tools 污染外部学习样本，签名机制帮助内部系统确认哪些行为链路可以被执行或

¹⁴ 视频来源区间：00:14:00-00:14:52。

信任。前者提高模仿者训练风险，后者提高自家执行链路的完整性。

反蒸馏与前面的 evaluation、ablation 并不割裂。evaluation 负责让产品持续变好，ablation 负责弄清机制贡献，anti-distillation 负责保护这些机制长期积累出的行为资产。对 Agent 公司来说，真正值钱的往往是一整套工程经验：真实任务、失败样本、修正策略、工具调用轨迹和评测结果共同形成行为资产。外部 API 调用如果能轻易把这些链路吸走，公司投入的实验成本就会被摊薄。

4.5 本章小结

本章把 Agent 产品研发讲成一套研究式产品工程。第一，Frontier lab 的产品迭代高度依赖 evaluation system；Manus 这类例子说明，完善测试用例和 prompt 版本流水线可以支撑每天多版本迭代。第二，A/B 实验和 ablation 要分清：A/B 实验比较真实流量中的产品版本，ablation 用开关拆出单项机制的贡献；GrowthBook 这类实验系统把分流、开关、指标和结果分析工程化。第三，反蒸馏保护的是行为链路和训练价值：fake tools 可以污染外部蒸馏训练集，CoT 摘要会降低完整推理链的可复制性，加密签名则指向更强的内部链路校验。评测让系统会进步，消融让团队知道为什么进步，反蒸馏让这些进步不被轻易搬走。

5 行业判断：GUI、Context 与 AI-native Organization

前几章讲的是 Agent 产品的内部工程：上下文、工具、记忆、安全、错误恢复、评测和反蒸馏。从 00:15:14 开始，讲者把视角拉到行业层面：当模型越来越强，软件界面、SaaS 价值、人的工作位置和组织协作方式都会被重新估价。这一段最容易被误读成未来主义口号；更准确的读法是，它在讨论一个朴素的工程问题：如果 Agent 的读写、搜索、调用和总结速度远高于人，哪些东西还应该服务人，哪些东西应该直接服务 Agent，哪些东西必须继续由人承担判断责任。

5.1 GUI 作为人的注意力补丁

讲者引用真格基金钟天杰的一篇文章，把 GUI 概括成“给人有限注意力打的补丁”（00:15:19–00:15:24）。这句话很狠，但它抓住了图形界面的底层约束：人类的视觉注意力、短时记忆、阅读速度和操作速度都有限，所以界面要把复杂系统切成按钮、菜单、列表、图表和当前屏幕。GUI 的伟大之处在这里，局限也在这里。

在 00:15:28–00:15:43，讲者说人阅读和思考的速度比 Agent 慢几十倍到几百倍，并指出一个人一次只能看到 PPT 上的几行字。这个观察并不需要精确到某个统一倍数，关键在于方向：GUI 把信息压成适合人的窄窗口；Agent 若被迫像人一样盯着窄窗口、点按钮、读少量可见文本，效率会被人类界面拖住。

GUI 的重新定位

GUI 的价值来自人类注意力稀缺：它把复杂状态压缩成可见、可点、可理解的局部画面。Agent 时代不会取消所有 GUI，却会减少“给 Agent 模拟人类看屏幕”的必要性。面向 Agent 的系统更需要结构化状态、可查询日志、清晰权限、机器可读 API 和可恢复的任务轨迹。

这也解释了讲者对 Claude Code 的判断：Claude Code 没有把重点放在传统 IDE 式 GUI 上，背后的产品假设是“人不需要一直看代码”（00:16:00–00:16:08）。这个假设有争议，因为 Cursor 这类 IDE 产品仍然服务大量“边看边改”的开发者；讲者也没有把 Claude Code 的增长全部归因

¹⁵ 视频来源区间：00:15:20–00:16:12。

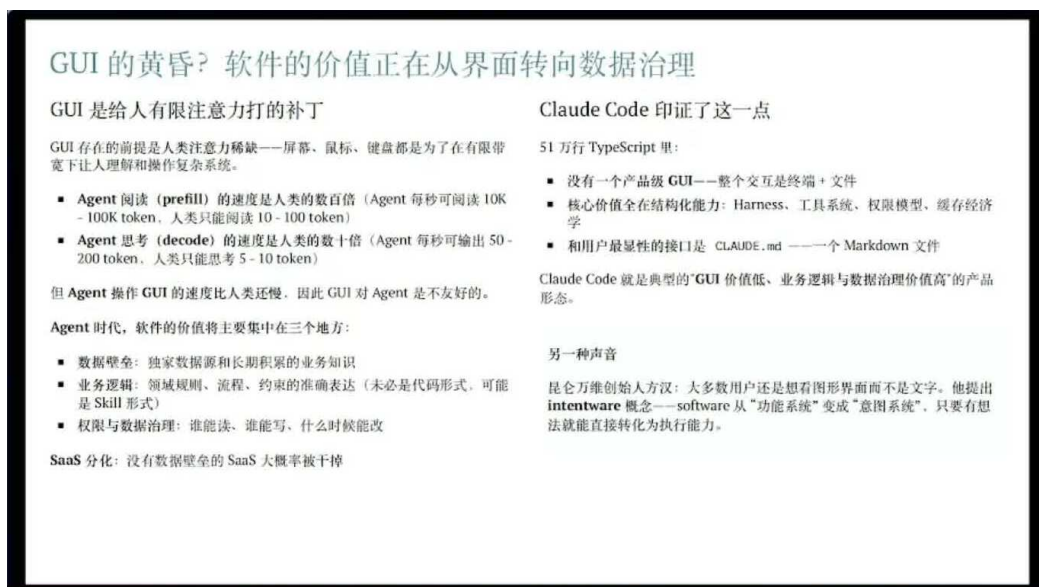


图 15: GUI 的价值正在从“给人看的界面”转向数据、业务逻辑、权限与数据治理: 幻灯片同时给出 Agent 读写速度、人类注意力瓶颈、Claude Code 低 GUI 假设和 SaaS 分化判断。¹⁵

于界面理念, 他补了一句商业因素: Claude Code 的 token 更便宜, 因为它使用 first-party token (00:16:16–00:16:20)。这个补充很重要。行业判断如果忽略成本结构, 就容易把产品胜负讲成纯粹哲学。

更具体地说, GUI 的下降发生在三类场景。第一, 重复性查看: Agent 可以直接读文件、索引、日志和工具返回, 不需要一页页翻。第二, 精密人工操作: 过去某些 SaaS 靠复杂表单和流程锁住用户, Agent 可以把表单操作变成 API 调用或浏览器自动化。第三, 代码细节查看: 基础模型足够强以后, 人可能只审阅关键 diff、设计边界和测试结果, 低价值逐行阅读会减少。这里的结论很硬: 如果一个产品的主要价值只是让人以很复杂的方式点击界面, 它的价值会被压缩。

“不用看代码”不等于免审查

讲者说人越来越少关心代码具体细节, 不能理解成工程师可以放弃审查。更合理的迁移是: 人少看样板代码、多看关键边界; 少看重复实现、多看权限、数据流、失败恢复、测试覆盖和业务后果。Agent 可以生成和修改大量代码, 责任仍会回到人对结果的评估。

5.2 软件价值的迁移: 数据、业务逻辑、权限和治理

紧接着, 讲者问了一个更根本的问题: 如果界面价值下降, 软件还剩什么价值? 他的答案在 00:16:32–00:16:40 给出: 数据、专用领域业务逻辑、权限和数据治理。这三个词比“AI 原生”“智能化”更朴素, 也更接近企业软件的真实护城河。

可以把软件价值粗略拆成四块来理解:

$$V_{\text{software}} \approx V_{\text{data}} + V_{\text{logic}} + V_{\text{governance}} + V_{\text{workflow}}$$

其中:

- V_{data} : 软件掌握的一方数据、历史记录、客户行为、交易状态和标注反馈。

- V_{logic} : 行业特定业务规则，例如计费、风控、供应链约束、医疗流程、财税口径或研发发布流程。
- $V_{\text{governance}}$: 权限、审计、合规、数据隔离、审批链和责任归属。
- V_{workflow} : 把上述三者组织成可执行流程的能力；这部分如果只依赖人工点界面，价值会被 Agent 自动化侵蚀。

这个分解能解释讲者对普通 SaaS 的悲观判断：没有数据壁垒，还靠一套软件让人“非常精密地操作”，大概率会被干掉（00:16:42–00:16:53）。这里的“干掉”指利润池会迁移，并非所有 SaaS 公司明天消失。过去用户为界面、流程和培训付费；之后用户更愿意为数据闭环、业务正确性、权限治理和可验证自动化付费。

为什么权限和治理会升值

Agent 执行动作的范围越大，权限和治理越值钱。读一个报表、改一个字段、发一封邮件、提交一段代码，在 UI 时代只是不同按钮；在 Agent 时代，它们对应不同风险等级、审计要求和回滚路径。软件若能清楚回答“谁允许 Agent 做什么、依据是什么、出了事怎么追溯”，就拥有真正的企业价值。

这也是为什么“做一个更好看的界面”不足以构成长期优势。界面仍然重要，尤其在管理、审计、解释和人类决策场景中；可一旦任务本身可以交给 Agent，软件层的核心就会从“把流程展示给人”迁移到“把状态、规则和权限可靠地交给 Agent”。

5.3 Context 决定人和 Agent 的可行动范围

00:16:57 之后，讲者转向很多人最焦虑的问题：人会不会被 AI 取代。他把核心变量从智商转向 context。这里要沿用第二章的定义，不重新定义 Agent：对 Agent 来说，context 包括它看见的任务状态、工具调用、工具返回、文件、记忆摘要和历史轨迹；对人来说，context 包括做过的事情、从中得到的经验、见过的失败、理解的隐性约束，以及一些还没有说出口的判断。

用符号写，第二章提到的状态 S_t 可以帮助读者抓住这件事：

$$S_t = \{\text{任务目标, 历史操作, 工具结果, 文件状态, 记忆摘要, 隐含约束}\}$$

其中：

- S_t : 时刻 t 的任务状态，也就是 Agent 或人做判断时实际拥有的上下文。
- 历史操作：已经执行过的命令、修改、沟通和决策。
- 工具结果：测试输出、日志、搜索结果、接口返回等外部反馈。
- 隐含约束：需求背后的业务、历史、组织、情绪和风险边界，很多时候没有被写进文档。

讲者举了一个 OpenAI RL researcher 的例子：对方觉得自己在 OpenAI 做的工作没有想象中难，也不一定需要极高智商；讲者说自己也有类似感受，真正决定一个人能做什么的，是他处在什么 context 里（00:17:25–00:17:43）。这句话听起来像鸡汤，落到工程里就很具体：同一个人如果没有训练数据、实验平台、内部讨论、失败日志和评测反馈，就很难做出同样质量的研究；同一个 Agent 如果看不到仓库历史、用户真实 bug、工具结果和权限边界，也很难完成同样任务。

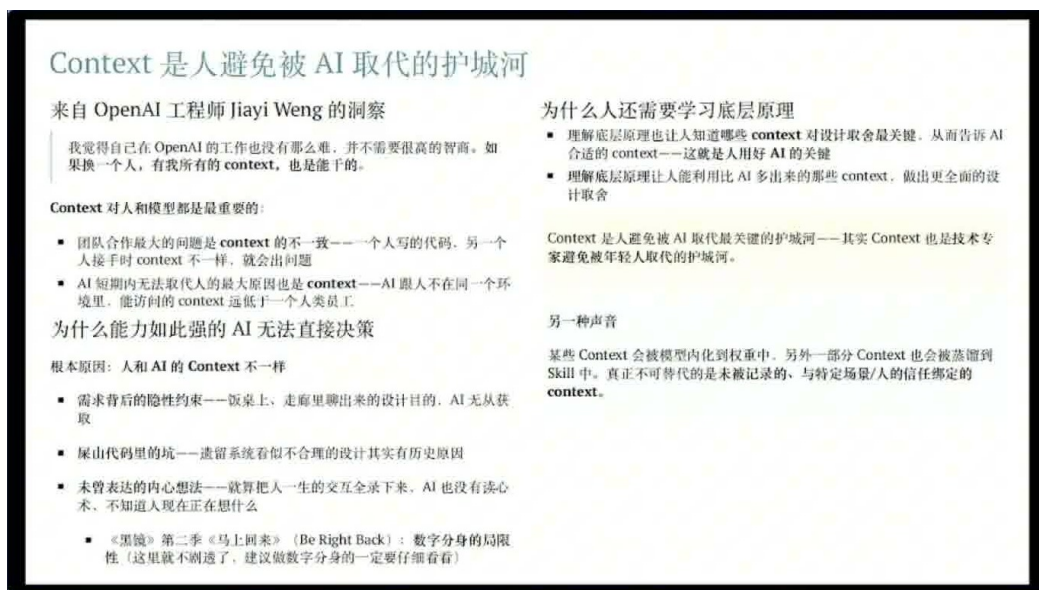


图 16: Context 被讲者视为人类避免被 AI 快速替代的护城河：画面把团队协作、隐性约束、历史代码原因、未表达想法和底层原理学习放在同一张判断框架中。¹⁶

讲者随后把 context 分成三类人的优势来源 (00:18:00–00:18:48)。第一，需求背后有大量隐性约束。产品经理一句“做个导出功能”，背后可能有客户合同、审计要求、字段口径、历史性能事故和老板没有明说的政治风险。第二，历史代码背后有历史原因。讲者提到 Claude Code 源码里有大量注释，很多注释标着 case number，表示某个现实生产问题曾经浪费几十万 token，于是工程师把一个 fix 加到了那里 (00:18:14–00:18:28)。第三，人有未表达的内心想法。即使把一个人与世界交互的所有历史都记录下来，也没有读心术，系统仍然不知道这个人当下真正担心什么、愿意承担什么风险、准备如何取舍。

人的价值落在 context 与判断

人比 Agent 多出来的价值，主要落在 context 选择与结果评估：知道该给 AI 哪些 context，知道如何把隐性约束转成明确要求，也知道如何用 common sense 判断结果能否进入现实流程。

这也回答了“有了 AI 还要不要学习底层原理”的问题。讲者明确反对“以后不用上学”的想法 (00:18:52–00:19:17)。底层原理的价值不只在于手写实现，更在于压缩和筛选 context。懂数据库的人更知道该告诉 Agent 哪些一致性约束；懂安全的人更知道哪些输出不能外发；懂编译和系统的人更容易看出性能瓶颈；懂业务的人更能判断一个看似正确的自动化结果会不会破坏实际流程。

5.4 AI-native organization、Brooks 定律与概念完整性

顺着 context sharing，讲者把讨论推进到组织形态。他观察到 Pine AI 自己，以及他了解的 Anthropic、Kimi 等公司，都在构建 AI-native organization：用 AI 改造信息流、协作方式和 ownership，减少传统组织里“上传下达”的损耗 (00:19:18–00:19:36)。

这里需要补一段软件工程背景。Brooks 在《人月神话》中提出过一个著名判断：向已经延期的软件项目继续加人，常常会让项目更晚。原因通常不在新人能力本身，更多在沟通路径增加、上下

¹⁶ 视频来源区间：00:17:05–00:18:20。

文同步成本暴涨、系统设计意图被稀释。Brooks 还强调“概念完整性”：一个系统应该体现统一的设计意图，像一座建筑由同一套空间逻辑组织，而不该像委员会把各自喜欢的房间拼在一起。

讲者说 AI 给这个老问题提供了一个新答案 (00:19:38–00:20:15)。过去，历史上很多优秀软件从一个人做的 MVP 起步，因为一个人最容易保持概念完整性。AI 让小团队或个人在早期阶段不必雇一大堆人开发 MVP，仍然能覆盖前端、后端、infra、Agent 等多个层面。更关键的是，AI-native 组织如果足够扁平，每个人都有完整 ownership，从零到一负责一个完整东西，就能减少“一个人只做前端、一个人只做后端、一个人只做 infra、一个人只做 Agent”带来的割裂。

AI Native 组织：用 AI 替代传统的上传下达

Brooks 的《人月神话》有了新答案

Brooks 指出：沟通开销是软件项目最大的敌人——给延期项目加人只会更慢。他的解法是“概念完整性”——系统应反映单一架构师的统一心智模型。

软件工程历史上最好的软件，在 MVP 阶段基本都是单个技术专家亲自动手实现的（UNIX、C、Linux、Git 等），“委员会设计”产出的软件几乎一定是平庸的。

AI 可以更好地实现概念完整性——不是因为协调变好了，而是协调变得不必要了：

指标	单人 × AI 数据
日均代码产出	4-5 万行
日均 Token 消耗	18 亿
单日最高 Git Commit	1,374 次

AI Native 公司的共同特征

- 扁平结构：Anthropic、Kimi 等公司的中层大幅压缩。上传下达被 AI 替代。
- Context 共享：Jiayi Weng (OpenAI) ——“人类组织的千古难题：难以保持组织架构的 context sharing 一致性”。

“砍掉高层的手脚，砍掉中层的屁股，砍掉基层的脑袋”

这句话子在现在的 AI Native 公司不再成立了。

传统中层的核心职能是上传下达。这三件事 AI 都能做，且比人做得好——因为 AI 不会为了公司政治扭曲信息。

所以 AI Native 组织不是砍掉一半人，而是把中层这个信息上传下达功能交给 AI，让高层直接看到基层信号、让基层直接看到战略 context。

另一种声音：给传统公司的务实建议

昆仑万维创始人方汉：很多国内公司还没完成信息化——传统公司不要当先烈，也不要成为后进生。

- 从上到下考 AI（笔试+机试），让组织人知道 AI 的边界在哪。
- 已成熟的（chatbot, AI coding）全力推进，其他场景保持克制。

图 17: AI-native organization 的关键在于 context sharing 与 ownership：幻灯片把 Brooks 的概念完整性、小团队 MVP、扁平组织和“上传下达”替代问题连接起来。¹⁷

概念完整性的直觉

概念完整性可以类比成一部电影的导演意识：摄影、剪辑、配乐、表演都可以由不同专家完成，但整体必须服务同一个叙事意图。软件也是如此。前端交互、后端模型、权限系统、数据结构和 Agent 行为如果各自为政，用户会感到系统“拼”出来了，任务流也会到处漏风。

讲者对传统大公司软件效率差的批评也落在这里：如果分工过细，最后产物容易变成“四不像”，也就是所谓委员会设计 (00:20:15–00:20:29)。AI-native organization 的潜在改进是两条。第一，执行层工作可以更多交给 AI，从而让人保留更完整的问题所有权。第二，中层大量“上情下达、下情上报”的摘要工作可以由 AI 总结，让人直接查询同事 Agent 的 context。

讲者给了一个公司内部协作例子：他们很少开会，想了解同事进展时，直接问对方的 Agent，因为那个 Agent 有对方工作的 context (00:20:51–00:21:10)。这个例子很重要：它没有把 AI-native organization 降格成“大家都用聊天机器人”，它指向一种 context 可查询结构。进展、阻塞、决策、文件、任务轨迹沉淀在 Agent 可访问的状态里，人不必通过会议反复搬运信息。

¹⁷ 视频来源区间：00:19:28–00:20:28。

AI-native 组织不能简化成少开会口号

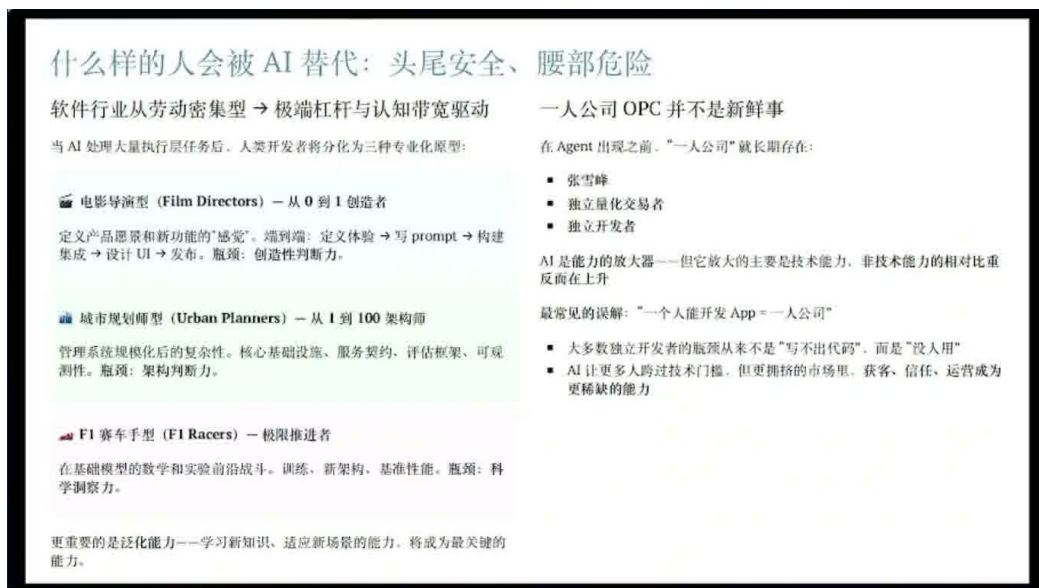
少开会只是表象。真正的变化在于 context 是否被结构化沉淀，ownership 是否足够清晰，Agent 是否能代表一个人的工作状态被查询，决策和风险是否可追溯。缺少这些条件，减少会议只会制造新的信息盲区。

5.5 三类更稳固的人类专业原型

00:21:10 以后，讲者给出一个职业判断：头尾相对安全，腰部更危险。这里的“尾部”指和物理世界深度集成的工作，现实世界本身提供了复杂约束，例如安装、维护、医疗护理、制造现场、销售关系和线下服务；“头部”指做 top 的工作，也就是定义问题、推极限、做原创判断。危险的“腰部”是大量可被总结、分发、执行和监督的中间层工作。

讲者进一步提炼了三类更有价值的专业原型（00:21:21-00:21:40）。

- 电影导演型：从零到一的创造者。它的核心不在亲手完成每个镜头，在提出整体意图、选择素材、调度资源、判断取舍，并让作品形成统一气质。
- 架构师型：从一到一百的系统建造者。它的价值在于把早期原型变成可扩展、可维护、可治理的结构，尤其擅长处理旧系统、依赖关系和长期演化。
- 研究者型：推极限的人。它的工作是探索还没有稳定范式的问题，发现新方法、新边界和新评测。



什么样的会被 AI 替代：头尾安全、腰部危险

软件行业从劳动密集型 → 极端杠杆与认知带宽驱动

当 AI 处理大量执行任务后，人类开发者将分化为三种专业化原型：

- 电影导演型 (Film Directors) — 从 0 到 1 创造者**
定义产品愿景和新功能的“感觉”。端到端：定义体验 → 写 prompt → 构建集成 → 设计 UI → 发布。瓶颈：创造性判断力。
- 城市规划师型 (Urban Planners) — 从 1 到 100 架构师**
管理系统规模化后的复杂性。核心基础设施、服务契约、评估框架、可观测性。瓶颈：架构判断力。
- F1 赛车手型 (F1 Racers) — 极限推进者**
在基础模型的数学和实验前沿战斗。训练、新架构、基准性能。瓶颈：科学洞察力。

更重要的是泛化能力——学习新知识、适应新场景的能力。将成为最关键的能力。

一人公司 OPC 并不是新鲜事

在 Agent 出现之前，“一人公司”就长期存在：

- 张雪峰
- 独立量化交易者
- 独立开发者

AI 是能力的放大器——但它放大的主要是技术能力，非技术能力的相对比重反而在上升

最常见的误解：“一个人能开发 App = 一人公司”

- 大多数独立开发者的瓶颈从来不是“写不出代码”，而是“没人用”
- AI 让更多人跨过技术门槛，但更拥挤的市场里，获客、信任、运营成为更稀缺的能力

图 18: 讲者将职业风险压缩为“头尾安全、腰部危险”，并提炼三类更稳固的专业原型：电影导演型、城市规划师型和 F1 赛车手型，同时提醒一人公司 OPC 早已存在。¹⁸

这三类原型共同点很明显：它们都需要强 context，且都要求人能把模糊问题压成可行动方向。导演型要知道观众、叙事、资源和时机；架构师型要知道历史债务、迁移路径、边界条件和组织能力；研究者型要知道已有方法为什么到头了，以及下一步试什么。

¹⁸ 视频来源区间：00:20:55-00:22:12。

纯执行层的窗口会变窄

讲者的判断很不客气：如果一个人没有自己的想法，没有泛化能力，也没有学习新知识的能力，纯执行层工作可能在三到五年内很快被淘汰（00:21:40–00:21:52）。这不意味着所有执行都会消失；重复执行的议价能力会下降。人需要把执行经验升级成判断、抽象、协作和评估能力。

这里还要避免一个常见误读：所谓“头部安全”并不保证高层管理者天然安全。传统中层如果主要工作是同步信息、催进度、写总结、转述需求，也会被 AI 压缩。所谓“尾部安全”也不保证低技能岗位安全；物理世界、客户关系和现实责任会让自动化变慢。真正的安全来自不可轻易复制的 context、责任承担和判断质量。

5.6 一人公司的误区

最后，讲者谈到“一人公司”。他的态度很清楚：AI 让一人公司更友好，但一人公司并非 AI 出现之后才有的新鲜事；最常见误解是“一个人能开发出 App 就等于一人公司”（00:21:56–00:22:14）。这句话需要说透，因为市场上很多讨论把“能写代码”误当成“能经营公司”。

公司是一套持续创造和捕获价值的系统，至少包括产品定义、获客、销售、交付、支持、财务、法务、合规、数据治理、品牌信任和现金流管理。App 是其中一个资产，甚至只是最容易被 AI 加速的一块。一个人用 Agent 做出漂亮 App，只证明技术生产门槛下降；他是否能拿到客户、处理退款、维护数据安全、承担故障责任、形成持续收入，是另一组问题。

一人公司的真实门槛

AI 会降低局部生产成本，同时提高技术能力在小团队中的占比。过去一个人可以把非技术工作外包给团队；一人公司场景下，创始人反而更需要理解技术、数据、权限、自动化和风险，因为这些东西会直接决定公司能不能持续运行。

这也是讲者说“技术能力的占比反而会上升”的原因（00:22:14–00:22:17）。当 Agent 帮一个人覆盖更多岗位时，这个人要承担更多系统设计责任。界面可以少一点，组织可以小一点，执行可以自动一点；可是数据边界、权限治理、业务逻辑、客户承诺和失败恢复都不会自动消失。能做 App，只是拿到了创业的第一个零件。

这一章因此把行业判断收束到一个更具体的方向：GUI 价值迁移，软件价值向数据、逻辑和治理集中；人的价值来自 context 和评估；组织价值来自更好的 context sharing 和 ownership；职业价值来自创造、架构和研究；一人公司仍然要面对完整经营系统。下一章会把这些讨论过渡到那句核心压缩： $\text{Model} \times \text{Harness} = \text{Agent}$ ，但公式的机制留给下一章正式展开。

5.7 本章小结

本章讨论了 Agent 时代的几组行业迁移。第一，GUI 是服务人类注意力的补丁，Agent 若被迫沿用人类 GUI，会浪费它在读写、搜索和调用上的速度优势。第二，软件价值不会消失，会迁移到数据、专用业务逻辑、权限和数据治理；没有数据壁垒、只靠复杂人工操作的 SaaS 会承压。第三，人的核心价值来自 context：需求背后的隐性约束、历史代码里的 case number、未表达的判断和最终评估能力。第四，AI-native organization 的重点是 context sharing 和 ownership，Brooks 所说的概念完整性在小团队和 Agent 辅助开发中变得更关键。第五，更稳固的人类专业原型是导演型创造

者、架构师型系统建造者和研究者型探索者；纯执行层如果缺少想法、泛化和学习能力，会很危险。第六，一人公司不能简化为“一个人写出 App”；它要求一个人借助 Agent 承担更完整的经营、技术和治理责任。

6 核心公式：Model × Harness = Agent

前面五章把 Claude Code 的源码细节一路拆到上下文、记忆、安全、错误恢复、评测体系、反蒸馏、GUI 价值迁移和组织协作。到了 00:22:21，讲者把这些细节压缩成一句话：如果只记住一个公式，他希望读者记住的是 Model 乘以 Harness 等于 Agent。这句话的价值不靠形式漂亮；它把 Agent 的能力来源从“模型加几个工具”的朴素想象，拉回到模型能力与外围工程系统的协同优化。

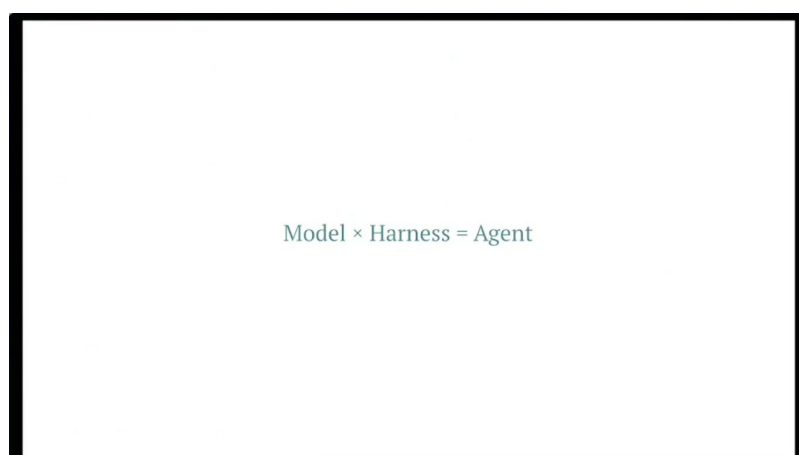


图 19: 核心压缩公式：Model × Harness = Agent。画面用极简公式把模型能力与外围工程系统的乘法关系钉住，强调 Agent 的交付质量来自二者协同。¹⁹

6.1 公式的含义：协同优化

讲者在 00:22:29–00:22:55 特意解释，为什么这里说“乘以”而非简单相加。直觉很简单：一个 Agent 的可用性像一辆车的真实通行能力，发动机再强，如果方向盘、刹车、轮胎、路况感知和维修系统跟不上，车开不远；外围系统再精密，如果发动机本身没有足够动力，也只能把有限能力包得更稳。模型与 Harness 彼此限制，也彼此放大。

讲者在 00:22:38–00:22:55 特别把这个公式放回行业语境里：他提到 Kimi 较早提出“模型 Agent”概念，但外界常把它理解成“模型加几个工具”。这个理解会漏掉关键部分。所谓模型 Agent，更接近模型能力与外围 Harness 的协同优化：模型提供推理和生成底座，Harness 负责上下文、工具、权限、验证、纠错、回流和产品化运行。工具只是动作空间的一部分，不能代表完整 Agent。

$$\text{Model} \times \text{Harness} = \text{Agent}$$

- Model：基础模型能力，提供推理、生成、代码理解、工具选择和语言表达的原始智能来源。
- Harness：模型外围的工程系统，包括上下文组织、工具编排、权限、安全、验证、纠错、记忆、评测、错误恢复、数据回流和产品化流程。

¹⁹视频来源区间：00:22:21–00:22:28。

- Agent：可行动系统的整体表现，表现为它能否在真实任务中持续看见正确状态、选择合适动作、处理失败并交付结果。
- $\times$ ：乘法关系，表示两侧存在短板效应；任一侧明显薄弱，整体质量都会被压低。
- $=$ ：工程近似意义上的等号，强调 Agent 既不能还原为单个模型权重文件，也不能还原为工具列表；它是两类能力耦合后的系统。

为了把乘法直觉说得更清楚，可以再写成任务质量版本：

$$Q \approx M \times H$$

- Q ：Agent 在真实任务中的质量，例如完成率、正确性、可恢复性、安全性和用户信任。
- M ：模型能力的强度，包含基础推理、代码能力、规划能力和泛化能力。
- H ：Harness 成熟度，包含工程化约束、验证、纠正、回滚、评测和数据闭环。
- $\approx$ ：近似关系，提醒读者这属于工程判断框架，不能当作物理定律。

乘法公式的工程含义

强模型配弱 Harness，会在权限、安全、记忆、工具调用、错误恢复和实验迭代上掉链子；强 Harness 配弱模型，会把流程做得规整，却仍然被推理能力和生成质量卡住。真正的 Agent 产品要同时优化 M 和 H ，并让真实用户问题在两者之间形成反馈。

这里要特别压住一个常见误读：把 Agent 理解成“模型 + 几个工具”会漏掉生产系统最重的部分。工具只是动作空间的一部分；Harness 还要决定什么时候调工具、怎么验证工具返回、什么时候要求用户确认、失败后如何续跑、哪些 bug 进入模型训练、哪些问题先留在产品层兜底。00:22:55–00:23:03 这一段的重点正是：Claude Code 的工作方式远比极简工具壳复杂。

6.2 用户问题、应用层 bug、模型训练与 Harness 兜底

从 00:23:03 开始，讲者给出一个很具体的闭环：用户在 Claude Code 里遇到问题，这些问题先沉淀到应用层；应用层的 bug 和需求再被整理，其中一部分交给训练部门，另一部分由 Harness 先补上。这条链路把“产品 bug”变成了模型公司内部的学习材料。

可以把这个过程写成一条工程流水线：

$$B_t \longrightarrow \text{Triage}(B_t) \longrightarrow \{D_1^{\text{train}}, H^{\text{fallback}}\} \longrightarrow M_{t+1}, H_{t+1}$$

- B_t ：时刻 t 收集到的 bug backlog，也就是真实用户在产品里遇到的问题集合。
- $\text{Triage}(B_t)$ ：问题分流过程，包括去重、归类、复现、优先级排序和根因判断。
- D_1^{train} ：进入训练或后训练流程的一方数据子集，来自产品自己的用户交互和故障记录。
- H^{fallback} ：暂时放在 Harness 里的兜底逻辑，例如特殊规则、验证器、重试策略、补丁流程或权限判断。

²⁰ 视频来源区间：00:23:03–00:23:13。

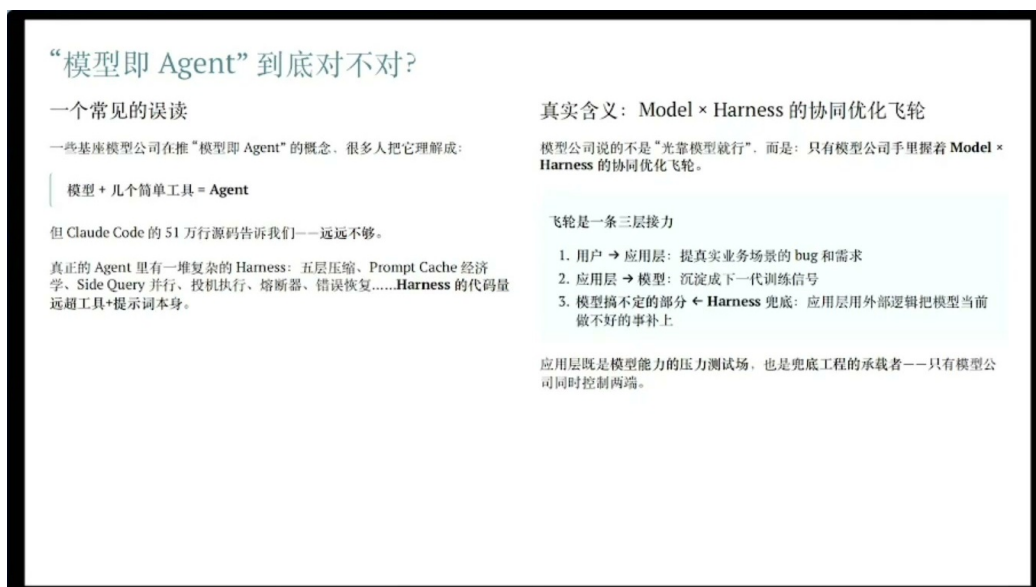


图 20: 协同优化飞轮：用户问题进入应用层，应用层将 bug 与需求分流给模型训练与 Harness 兜底，形成一方数据驱动的反馈闭环。²⁰

- M_{t+1} ：吸收部分问题后的下一轮模型能力。
- H_{t+1} ：吸收短期兜底和工程修复后的下一轮 Harness。

00:23:18–00:23:30 里，讲者提到基础模型公司会把用户看到的 bug 列出来，甚至可能是几千个 bug，然后把其中一部分汇总给训练部门。这里最值得抓住的是“其中一部分”。训练数据不能当垃圾桶，不能把所有线上坏例子直接塞进模型；需要判断哪些问题反映模型能力缺陷，哪些问题来自工具设计、上下文组织、权限策略、输出格式、UI 提示或外部服务不稳定。

Bug backlog 为什么不能直接当训练集原料

真实 bug 进入训练前通常要经过复现、清洗、标注、归因和优先级排序。一个“Claude Code 没有完成任务”的用户反馈，可能对应模型推理失败，也可能对应工具返回超时、上下文被压缩坏了、权限确认流程阻塞、文件系统状态读错、评测用例缺失。分不清根因，训练会把产品问题误喂给模型。

这也解释了 00:23:32–00:23:39 的那句提醒：训练模型没有阿拉丁神灯效果。训练部门拿到 bug 汇总后，不能立刻把模型训成完美版本。训练有数据规模、标注质量、算力排期、评测回归、灾难性遗忘和产品发布节奏等限制。于是，短期内解决不了的问题会留在 Harness 里，由产品工程先兜底。

兜底不能等同于低级补丁。它可以很粗糙，例如某个 case number 对应的硬编码保护；也可以很系统，例如错误恢复状态机、语义命令解析器、权限风险分级、输出校验器、工具调用重试队列、上下文压缩策略。关键看它是否被记录、被评测、被持续清理。如果兜底逻辑没有进入评测和回流，它就会变成没人敢动的工程债。

Harness 兜底的双刃剑

Harness 可以让产品在模型尚未解决某类问题时先可用，也会把短期例外堆成长期复杂度。健康的兜底逻辑必须带来源、触发条件、预期失效时间和回归测试；缺少这些信息，后续工程师只会看到一堆不敢删除的特殊分支。

6.3 First-party data 与数据飞轮

00:23:47–00:23:57 是本节的第二个核心：讲者说这是一种数据飞轮，首先公司有自己的 first-party data，其次它不断把能解决的问题逐渐内化到模型。First-party data 指产品自己直接产生和拥有的一方数据，例如用户真实任务、失败轨迹、工具调用序列、修复后的正确操作、用户确认和拒绝、回滚记录、评测回归结果。它的价值在于贴近真实分布。

第三方公开数据能教模型一般知识；一方数据能告诉系统“用户在这个产品里真正想做什么、常在哪里失败、哪些失败最贵、哪些错误可以由 Harness 修正、哪些错误必须提升模型”。这就是飞轮的起点：真实使用带来问题，问题带来归因，归因带来训练数据和 Harness 修复，修复后的产品吸引更多真实使用。

Usage $\rightarrow D_1 \rightarrow$ Bug Triage $\rightarrow$ {Model Update, Harness Update} $\rightarrow$ Better Agent $\rightarrow$ More Usage

- Usage：真实用户使用，产生任务轨迹、成功案例和失败案例。
- D_1 ：first-party data，一方数据，来自产品自身而非外部抓取。
- Bug Triage：对问题进行分流，判断进入模型、Harness、文档、产品流程或人工运营。
- Model Update：通过训练、后训练、偏好数据或评测修正提升 M 。
- Harness Update：通过工具、权限、记忆、恢复、校验和流程修正提升 H 。
- Better Agent：更可靠的 Agent，体现在完成率、安全性、稳定性和长任务连续性上。
- More Usage：更高频、更复杂的真实任务，继续产生下一轮数据。

数据飞轮的核心不等于“数据越多越好”

数据飞轮的关键在于闭环质量：真实任务能被记录，失败能被归因，归因能进入训练或 Harness，修复能被评测验证，验证后还能回到用户使用。只有数据堆积，没有归因和回归测试，飞轮会变成日志仓库。

这里也能看出基础模型公司和纯应用公司的差异。基础模型公司可以把一部分产品数据内化进模型能力；应用公司若没有模型训练入口，也可以把数据内化进 Harness，例如更好的任务模板、行业规则库、权限策略、工作流恢复和评测样例。两者的共同点是：一方数据越贴近真实任务，越能减少“实验室里很好、上线就出事”的落差。

6.4 Harness 化石与长周期任务的新问题

00:23:59–00:24:08，讲者用了一个很形象的说法：当一些问题逐渐被内化到模型里，Harness 里剩下的兜底逻辑会慢慢变成“化石”。化石不等于垃圾。化石记录了上一代模型能力边界和上一阶段产品事故：某个特殊 parser、某个权限确认弹窗、某个重试条件、某段 case number 注释，可能都来自一次真实失败。问题在于，当模型能力提升后，这些逻辑可能已经过时，却仍然影响新系统行为。

可以把 Harness 化石理解成工程里的沉积岩：每一层都来自一次事故、一次用户抱怨、一次评测失败或一次线上救火。沉积层有历史价值，但如果没有地质图，后来者只能在上面继续施工，越修越厚。健康的 Agent 工程要定期追问：这个兜底还有效吗？它覆盖的问题已经被模型解决了吗？它会不会阻挡更通用的新能力？它有没有评测能证明仍然必要？

化石化的真实风险

过时 Harness 会造成三类问题：第一，模型已经能处理的新任务被旧规则拦住；第二，特殊分支相互叠加，导致行为难以预测；第三，评测只覆盖旧事故，新能力上线后没有对应观测。化石需要被标注和考古，不能只靠“看起来还能跑”继续保留。

但讲者马上补了一句更现实的话：旧的“屎山”会减少，新的“屎山”也会出现(00:24:08–00:24:21)。原因很直接：模型越强，用户越会把更长、更复杂的任务交给它。昨天用户希望它改一个函数；今天希望它完成一天的工作；明天可能希望它连续跑一周，处理需求澄清、代码修改、测试、部署、反馈和二次修复。任务跨度一拉长，新问题就会浮出水面。

长周期任务的难点主要有五类：

- 状态漂移：一天或一周的任务里，文件、依赖、用户目标和外部环境都会变化，旧上下文可能误导后续动作。
- 记忆污染：Agent 需要压缩历史，但压缩会丢细节，也可能把错误结论写进长期记忆。
- 权限续期：短任务的一次确认不等于长任务全程授权，尤其涉及外发、支付、部署、删除和敏感读取时。
- 失败恢复：长任务更容易遇到工具卡死、网络失败、输出中断、测试环境变化和供应商限流。
- 责任归属：任务越长，越需要清楚记录每一步为什么执行、谁批准、失败后如何回滚。

Long-horizon task 与 long-horizontal task

字幕里的“long-horizontal task”更可能指 long-horizon task，即长时间跨度、多阶段、需要持续状态管理的任务。它的核心难度不止是单次推理长度，更在于跨时间保持目标、状态、权限、记忆和恢复能力。换成数学直觉，就是 S_t 到 S_{t+k} 的状态转移链更长，任意一步的小错都会被后续步骤放大。

因此，模型提升并不会让 Harness 消失。更准确地说，模型提升会改变 Harness 的压力位置：一部分旧兜底会沉降成化石，一部分旧规则会被模型能力替代，同时更长周期、更高风险、更强自治的新任务会产生新的 Harness 需求。Agent 工程的长期工作，就是不断把能内化的问题交给模型，把暂时不能内化的问题工程化兜底，再定期清理已经失效的兜底层。

6.5 本章小结

本章正式展开了全片的压缩公式： $\text{Model} \times \text{Harness} = \text{Agent}$ 。乘法强调短板效应：模型能力 M 和 Harness 成熟度 H 任一侧薄弱，都会把 Agent 的真实任务质量 Q 压低。讲者在 00:23:03–00:23:13 用 Claude Code 的例子把这个公式落到机制上：用户问题先进入应用层，应用层 bug 再分流给模型训练和 Harness 兜底。

这一闭环的燃料是 first-party data。真实用户任务形成一方数据 D_1 ，bug backlog B_t 经过归因后，一部分进入训练，一部分留在 Harness 里变成短期补丁、验证器、权限策略或恢复机制。随着模型吸收可解决的问题，部分 Harness 逻辑会化石化；随着模型能做更长周期的任务，新的状态漂移、记忆污染、权限续期、失败恢复和责任归属问题又会出现。下一章讨论模型商品化和应用层护城河时，应当把本章的公式作为技术底座，而不要把它简化成模型价格或工具数量的竞争。

7 总结与延伸：模型商品化与应用层护城河

讲者在最后一分钟把整场演讲收束到一个更战略的问题：如果模型越来越接近，应用层公司还靠什么活？这个问题不能用一句“模型会商品化”打发，因为模型市场正在分层。00:24:23-00:25:23 的实质内容包含四个判断：顶尖模型仍有差距，中端模型正在趋同；同等智能的 API 成本会快速下降，这是讲者给出的经验判断；应用层公司的长期护城河要放在技术之外；Harness 在短期内仍然是应用层最重要的技术杠杆。

应用层公司的护城河需要在技术之外

观察一：Harness 里的“屎山”是飞轮的化石

为什么模型公司不把全部 bug 都喂给下一代模型，让 Harness 变薄？因为训练不是阿拉丁神灯：

1. 训练需要时间（数月）
2. 模型不能内化训练数据中所有的约束和偏好

所以 Harness 里的“屎山”其实反映了模型公司内部的模型团队和应用团队之间的张力：

- 模型稳定掌握的 → 下一代 Harness 可以去掉
- 模型还不稳的 → Harness 里继续兜底

Claude Code 泄露源码里布满注释，记录着真实业务场景里模型没扛住、Harness 补上的一次次事件，是模型当前能力边界的化石。

观察二：模型会商品化吗？

顶尖模型：没有商品化，差距还会拉大

- 硅谷 AI 人才薪酬 \$1.5M-\$10M+，是同级别 AI 工程师的 3-4 倍
- 反蒸馏工程的存在 = 顶尖模型的推理轨迹还有独占价值
- 人才溢价 + 算力壁垒 + 数据飞轮——差距可能继续扩大而非收敛

中端模型（普通人类智力，日常工作）：正在商品化

- 开源和闭源前沿差距已在一年以内（Kimi K2.6 编码接近 Claude 4.6 Sonnet）
- 目前前沿模型的智力已经能胜任大多数人的日常工作，只缺 Context 和 Harness
- 同等智力的模型，API 价格一年下降超过一个数量级

关键启示：Harness 是应用层短期的技术杠杆，但技术优势最终会被模型公司的飞轮吃掉。应用层公司的长期护城河需要在技术之外：数据、渠道、牌照、用户信任、网络效应等等。

图 21：应用层护城河：同一页同时收束 Harness 化石、模型商品化、API 成本下降和长期护城河，说明短期技术杠杆与长期业务壁垒的分工。²¹

这一章不再重复前六章的细节，而把它们压成一条因果链：Claude Code 源码泄露暴露了生产级 Agent 的工程细节；这些细节聚合成 Harness Engineering；Harness 又延伸到安全、权限、错误恢复、evaluation、ablation 和反蒸馏；这些机制改变了软件价值、组织协作和人的位置；最后落到公式 $\text{Model} \times \text{Harness} = \text{Agent}$ 。到结尾处，问题变成：当 M 这一项变便宜、变接近时，应用层能不能用 H 、数据、业务和治理建立持续优势。

7.1 顶尖模型与中端模型的分化

讲者先给“模型会不会商品化”下了一个边界。他说商品化指的是模型之间差异变小，大家能力差不多，单个模型本身变得“不值钱”（00:24:23-00:24:32）。但他的判断更像一条分层曲线：顶尖模型之间仍可能保留能力差距，中端模型会更明显地走向商品化（00:24:34-00:24:43）。

这里的“商品化”可以类比云服务器。最顶级的芯片、网络和调度系统仍然稀缺；可对大量普通业务来说，算力实例越来越像标准零件，用户更关心价格、稳定性、接口和生态。模型也类似：frontier model 继续争夺最高推理、长程任务、工具使用、代码能力和多模态能力；中端模型在常见编码、问答、摘要、客服、数据处理等任务上逐渐接近，价格和可获得性会压缩差异。

讲者在 00:24:43-00:24:50 举了一个当时性的例子：刚发布的 Kimi K2.6 在编码能力上已经接近 Claude 4.6 Sonnet（字幕将 Claude 误写成 Cloud，将 Sonnet 误写成 Signite，按上下文校正）。这个例子不应被理解成精确 benchmark 结论；它的作用是说明一个趋势：对许多应用层任务来说，“足

²¹ 视频来源区间：00:24:23-00:25:23。

够好”的模型候选越来越多，模型选择会从绝对能力比较，转向能力、成本、延迟、上下文长度、工具协议、合规和稳定性的综合取舍。

紧接着在 00:24:52-00:25:05，讲者给了应用层公司的定位：一些 frontier model 的智能已经足以胜任大多数人的日常工作，剩下的缺口通常不只是“再聪明一点”。这些缺口可能是权限如何交付、数据如何接入、任务如何恢复、结果如何审计、客户场景如何落地、成本如何控制。应用层公司当前主要做的，正是把这种接近通用能力的模型放进具体业务现场，让它补齐最后一段从“会做”到“可交付、可追责、可持续使用”的距离。

模型商品化的准确读法

模型商品化指中端能力趋同、价格下降、替换成本降低。它不会自动抹平 frontier model 的上限差异，也不会让应用层工程失去价值。相反，模型越像可替换零件，应用层越需要把真实任务、权限、记忆、评测和数据回流组织成难以复制的系统。

7.2 API 成本下降规律与短期技术杠杆

讲者随后提到一个经验判断：同等智能模型的 API 成本大约每年下降一个数量级（00:25:05-00:25:13）。这里要谨慎表述。这条说法不能当成物理定律，也不能当成覆盖所有供应商、所有模型、所有任务的硬承诺；它更像演讲中的行业经验压缩，表达的是“同等能力会快速变便宜”。

如果用符号写，可以把讲者的判断表达为：

$$c_{\text{API}}(y+1) \approx 0.1 c_{\text{API}}(y)$$

- $c_{\text{API}}(y)$ ：第 y 年达到某一等效智能水平的 API 调用成本。
- $c_{\text{API}}(y+1)$ ：下一年达到同一等效智能水平的 API 调用成本。
- 0.1：讲者口中的“下降一个数量级”，表示约为上一年的十分之一。
- $\approx$ ：近似关系，强调这是经验判断和方向性假设，不能当作保证收益率。

这个式子的含义很直接：如果一个应用层公司的优势只是“它接入了一个当下很贵、别人暂时用不起的模型”，这个优势会随着价格下降被侵蚀。API 成本下降以后，更多竞争者可以调用相似模型，用户也会更容易迁移。技术领先仍然重要，可纯粹依赖模型调用成本差的窗口会很短。

这正是 Harness 的位置。讲者在 00:25:18-00:25:23 明确说，短期来看，Harness 肯定是应用层的技术杠杆。所谓杠杆，是用较少的模型能力变化换取较大的产品表现变化：同一个模型，如果多了 prompt caching、上下文压缩、side query、权限分类、错误恢复和评测回归，任务完成率会明显不同；同一个 API 价格，如果 Harness 把重复 token、失败重试和人工兜底降下来，单位任务成本也会变低。

短期杠杆与长期护城河的差别

Harness 是短期技术杠杆，因为它能迅速提升 Agent 的可靠性、成本效率和任务覆盖。长期护城河还需要更慢、更重的东西：一方数据、业务流程、客户关系、分发渠道、权限治理、审计责任、品牌信任和组织学习。前者像一台更好的发动机，后者像道路、仓库、车队调度和客户合同。

7.3 应用层护城河为什么还要放在技术之外

讲者的结论很硬：从 API 成本快速下降这个角度看，应用层公司的护城河一定还要放在技术之外 (00:25:13–00:25:18)。这句话容易冒犯工程师，因为它听起来像在贬低技术。更准确的解释是：单点技术会被学习、开源、内化或降价；能长期防守的，通常是技术和现实系统的绑定。

应用层护城河至少包含四类非单点技术因素。

- 一方数据：真实用户任务、失败样本、业务结果、人工修正、审计记录和长期偏好。第六章的数据飞轮讲的就是这件事：用户 bug 进入应用层，一部分反馈给模型训练，暂时无法内化的留在 Harness 里兜底。
- 业务逻辑：行业规则、流程约束、例外处理、客户合同、财税口径、研发规范和组织惯例。这些东西很少完整写在公开文档里，通常藏在历史故事和人的 context 中。
- 治理能力：权限分级、风险分数、外发审查、回滚、审计、合规和责任归属。Agent 能做的动作越多，治理越成为产品价值的一部分。
- 分发与信任：谁能触达客户、谁能嵌入日常流程、谁能承担失败责任、谁能让组织愿意把权限交给 Agent。

把这四类因素和前面的 Harness 放在一起看，应用层的真正目标远大于“套一个模型做界面”。它要把模型变成可进入业务现场的行动系统。行动系统需要看见正确状态，调用正确工具，在危险动作前停下来，在失败后恢复，用评测体系持续改进，并把真实世界反馈变成新的数据和规则。

最危险的应用层错觉

最危险的错觉是把护城河押在“当前模型效果更好”或“当前 prompt 更巧”上。模型会升级，prompt 会被模仿，API 会降价。更稳的壁垒来自真实任务闭环：客户每天把什么问题交给系统，系统如何记录失败，团队如何把失败转成评测、权限、记忆、工具和业务规则。

7.4 全文综合：从源码细节到组织形态

整场演讲的主线可以压成六步。

- 第一步，源码泄露提供了观察窗口。Claude Code 与 OpenCode 的差异，提示读者同一个模型接入不同外围工程后，可靠性会出现巨大差别。
- 第二步，Harness Engineering 给出解释框架。模型给出基础智能，上下文和工具决定能看见什么、能做什么，Harness 负责约束、验证、纠正和产品化运转。
- 第三步，生产级 Agent 要先处理下限。安全、敏感信息、权限分类、语义命令解析和错误恢复，决定系统会不会在真实 workflow 里闯祸。
- 第四步，产品进化需要实验系统。Evaluation 把失败样本固定下来，ablation 拆出机制贡献，A/B 实验检查真实流量，anti-distillation 保护行为链路和训练价值。
- 第五步，行业变化落到 context。GUI 的价值会迁移，软件价值集中到数据、业务逻辑和治理；人的价值来自隐性约束、历史原因、判断和最终评估；AI-native 组织的重点是 context sharing 和完整 ownership。

- 第六步，核心公式把一切合起来： $\text{Model} \times \text{Harness} = \text{Agent}$ 。当 M 强而 H 弱时，系统会在权限、记忆、恢复、评测和业务闭环上掉分；当 H 持续吸收真实 bug，它又会推动模型训练、产品演化和数据飞轮。

这条链的起点是代码泄露，终点已经超出代码。代码只是把工程细节暴露出来；真正的结论是，Agent 产品的竞争会从“谁接了更强模型”扩展到“谁能把模型放进更可靠的行动环境”。Harness 是这个行动环境的技术骨架，应用层护城河则是骨架之外的肌肉、神经和现实连接。

7.5 后续问题与实践清单

如果要审视一个 Agent 产品是否具备可靠 Harness，可以按下面这份清单逐项追问。

- 上下文压缩：长任务中的关键约束、工具结果和用户偏好有没有被保留下来？压缩后的摘要是否能被评测回归？
- 旁路查询：权限分类、记忆检索、标题生成、todo summary、风险判断是否全压在主循环里？有没有用 side query 分担小而确定的判断？
- 权限审查：系统是否区分读取、修改、外发、执行命令和花钱等动作？是否有 R 风险分数与 L 权限级别一类的分层？
- 错误恢复：工具漏调、供应商卡死、输出长度中断、任务中途失败后，系统能否断点续写、重试、降级或请求人工确认？
- 评测体系：真实失败样本有没有进入 evaluation suite？prompt、记忆、工具策略和恢复策略上线前是否跑回归？
- 消融实验：团队是否知道某项机制带来的 Δ 收益，还是只凭感觉把多个改动一起上线？
- 数据回流：用户 bug、人工修正、业务结果和成功轨迹有没有形成一方数据 D_1 ，并进入模型训练、Harness 兜底或产品规则？
- 组织协作：Agent 的 context 是否能被同事查询？ownership 是否清晰？重要决策、风险和失败是否可追溯？

这份清单也引出几个开放问题。第一，当模型继续增强，哪些 Harness 逻辑会被内化进模型，哪些会因为合规、审计和业务责任继续留在应用层？第二，API 成本下降会压缩哪些应用的利润池，又会释放哪些过去成本过高的自动化场景？第三，一方数据进入模型训练时，隐私、合规、客户信任和反蒸馏之间如何平衡？第四，AI-native 组织如果把 context 大量交给 Agent 承载，怎样避免新的信息孤岛、权限滥用和责任不清？

最终可以把这场演讲的收束说得更直白一些：模型会越来越强，也会有一部分越来越便宜；靠“调用一个强模型”创业的窗口会变短。应用层真正要做的，是把模型放进一个能看见真实状态、遵守权限、持续评测、吸收失败、沉淀数据、承担责任的系统里。短期看，这是 Harness Engineering；长期看，这是应用层护城河。